Facultatea de CIBERNETICĂ, STATISTICĂ ŞI INFORMATICĂ ECONOMICĂ
Specializarea: Informatică economică
Anul de studiu: 2

PROIECT SGBD - Oracle

Bază de date relațională

Student: Condurache-Bota Dora-Elena
Grupa: 1056
Profesor coordonator: Georgiana Nicolaie

An universitar:
2024 – 2025

1. Descrierea bazei de date:

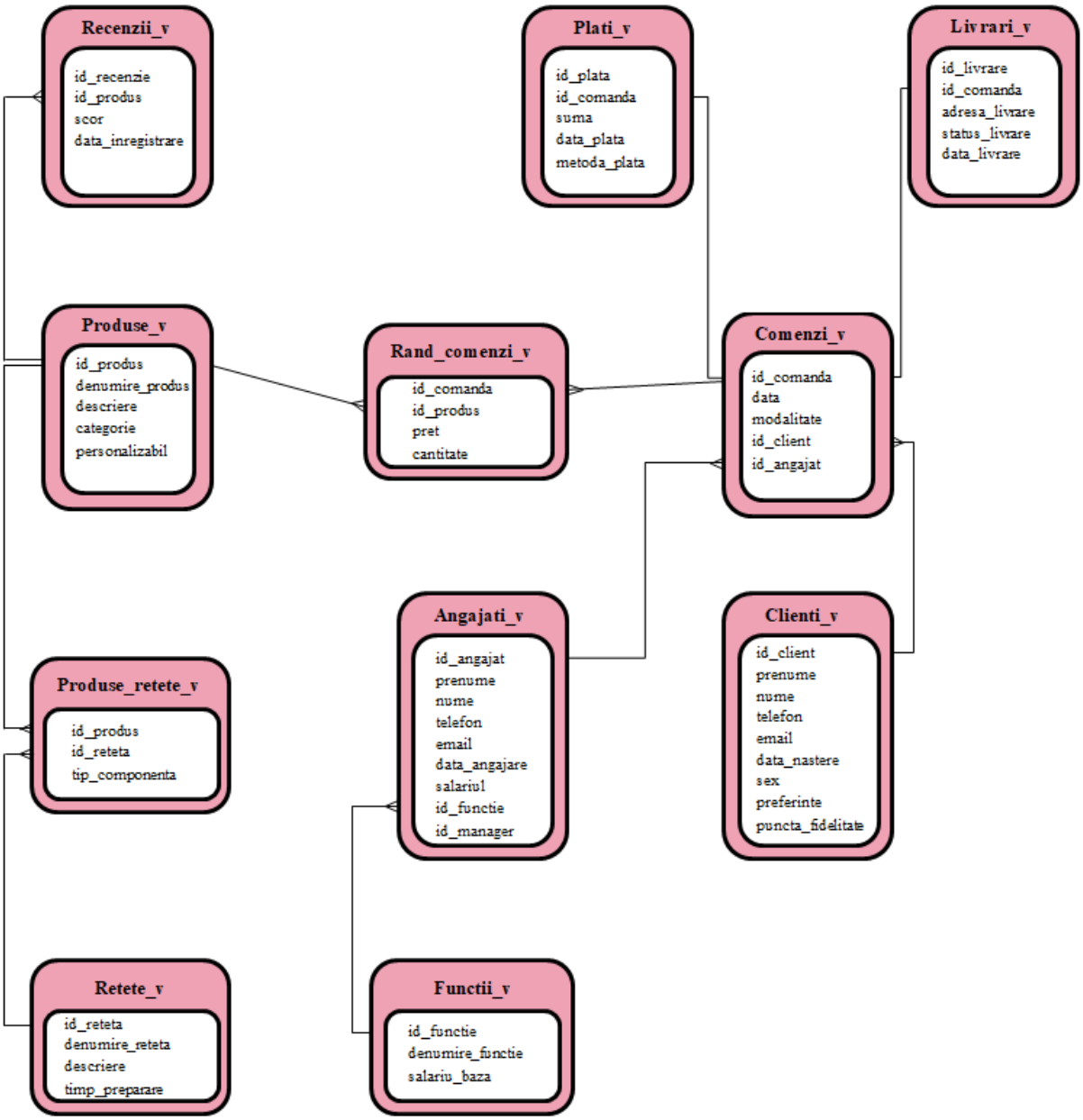
Baza de date urmărește gestionarea activității cofetăriei Vera, integrând informații esențiale despre clienți, angajați, produse, comenzi, rețete, plăți, livrări și recenzii. Structura bazei permite evidența detaliată a clienților, incluzând preferințe, date de contact și puncte de fidelitate. Angajații sunt înregistrați împreună cu informații despre funcție, salariu și ierarhie (manager), iar funcțiile sunt legate de salarii de bază.

Comenzile realizate de clienți sunt gestionate prin legături între tabelele comenzi_v, rand_comenzi_v și plati_v, permițând monitorizarea produselor comandate, valorii fiecărei comenzi și a statusului de plată. Produsele pot fi simple sau personalizabile, iar unele sunt asociate cu rețete compuse (prin retete_v și produse_retete_v), deoarece torturile au atât blatul, cât și crema făcută separat. Livrările sunt urmărite prin dată, adresă și status, iar clienții pot oferi feedback prin recenzii care includ scoruri și date de înregistrare.

Prin această structură relațională, baza de date permite nu doar stocarea eficientă a informațiilor, ci și automatizarea unor procese esențiale (de exemplu: validarea salariilor, acordarea de puncte de fidelitate, controlul prețurilor), susținând decizii corecte și rapide în administrarea cofetăriei.

- **Angajati_v**: id_angajat (primary key), prenume, nume, email, telefon, data_angajare, salariul, id_functie (foreign key), id_manager. Relație 1:M cu tabela Funcții.
- **Functii_v**: id_functie (primary key), denumire_functie, salariu_baza.
- **Clienti_v**: id_client (primary key), prenume, nume, telefon, email, data_nastere, sex (M/F), preferinte, puncte_fidelitate. Relație 1:M cu Comenzi.
- **Comenzi_v**: id_comanda (primary key), data, modalitate, id_client (foreign key), id_angajat (foreign key). Relație 1:M cu Rand_comenzi.
- **Rand_comenzi_v**: id_comanda, id_produs, pret, cantitate. Este o tabelă de legătură între Comenzi și Produse.
- **Produse_v**: id_produs (primary key), denumire_produs, descriere, categorie, personalizabil. Relație 1:M cu Rand_comenzi.
- **Produse_retete_v**: id_produs, id_reteta, tip_componenta. Este o tabelă de legătură între Rețete și Produse.
- **Retete_v**: id_reteta (primary key), denumire_reteta, descriere, timp_preparare. Relație 1:M cu Produse.
- **Plati_v**: id_plata (primary key), id_comanda (foreign key), suma, data_plata, metoda_plata. Relație 1:M cu Comenzi.
- **Livrari_v**: id_livrare (primary key), id_comanda (foreign key), adresa_livrare, status_livrare, data_livrare. Relație 1:1 cu Comenzi.
- **Recenzii_v**: id_recenzie (primary key), id_comanda (foreign key), scor, comentariu, data_recenzie. Relație 1:1 cu Comenzi.

2. Diagrama bazei de date:



3. Structura tabelelor

- Angajati_v:

```
Name          Null?    Type
------------- -------- ------------
ID_ANGAJAT    NOT NULL NUMBER(2)
PRENUME                VARCHAR2(20)
NUME                   VARCHAR2(25)
TELEFON                VARCHAR2(15)
EMAIL                  VARCHAR2(35)
DATA_ANGAJARE          DATE
SALARIUL               NUMBER(8,2)
ID_FUNCTIE             NUMBER(2)
ID_MANAGER             NUMBER(2)
```

- Functii_v:

```
Name              Null?    Type
----------------- -------- ------------
ID_FUNCTIE        NOT NULL NUMBER(2)
DENUMIRE_FUNCTIE           VARCHAR2(30)
SALARIU_BAZA               NUMBER(8,2)
```

- Clienti_v:

```
Name               Null?    Type
------------------ -------- ------------
ID_CLIENT          NOT NULL NUMBER(2)
PRENUME                     VARCHAR2(20)
NUME                        VARCHAR2(20)
TELEFON                     VARCHAR2(20)
EMAIL                       VARCHAR2(30)
DATA_NASTERE                DATE
SEX                         VARCHAR2(10)
PREFERINTE                  VARCHAR2(40)
PUNCTE_FIDELITATE           NUMBER(3)
```

- Comenzi_v:

```
Name        Null?    Type
----------- -------- ------------
ID_COMANDA NOT NULL NUMBER(2)
DATA                DATE
MODALITATE          VARCHAR2(10)
ID_CLIENT           NUMBER(2)
ID_ANGAJAT          NUMBER(2)
```

- Rand_comenzi_v:

```
Name        Null? Type
----------- ----- -----------
ID_COMANDA        NUMBER(2)
ID_PRODUS         NUMBER(2)
PRET              NUMBER(8,2)
CANTITATE         NUMBER(8,2)
```

- Produse_v:

```
Name              Null?    Type
----------------- -------- ------------
ID_PRODUS         NOT NULL NUMBER(2)
DENUMIRE_PRODUS            VARCHAR2(30)
DESCRIERE                  VARCHAR2(70)
CATEGORIE                  VARCHAR2(20)
PERSONALIZABIL             VARCHAR2(2)
```

- Produse_retete_v:

```
Name              Null? Type
----------------- ----- ------------
ID_PRODUS               NUMBER(2)
ID_RETETA               NUMBER(2)
TIP_COMPONENTA          VARCHAR2(20)
```

- Retete_v:

```
Name              Null?    Type
----------------- -------- ------------
ID_RETETA         NOT NULL NUMBER(2)
DENUMIRE_RETETA            VARCHAR2(30)
TIMP_PREPARARE             NUMBER(3)
```

- Plati_v:

```
Name          Null?    Type
------------- -------- ------------
ID_PLATA      NOT NULL NUMBER
ID_COMANDA             NUMBER
SUMA          NOT NULL NUMBER(10,2)
DATA_PLATA             DATE
METODA_PLATA           VARCHAR2(50)
```

- Livrari_v:

```
Name           Null?    Type
-------------- -------- -------------
ID_LIVRARE     NOT NULL NUMBER
ID_COMANDA              NUMBER
ADRESA_LIVRARE NOT NULL VARCHAR2(200)
STATUS_LIVRARE          VARCHAR2(50)
DATA_LIVRARE            DATE
```

- Recenzii_v:

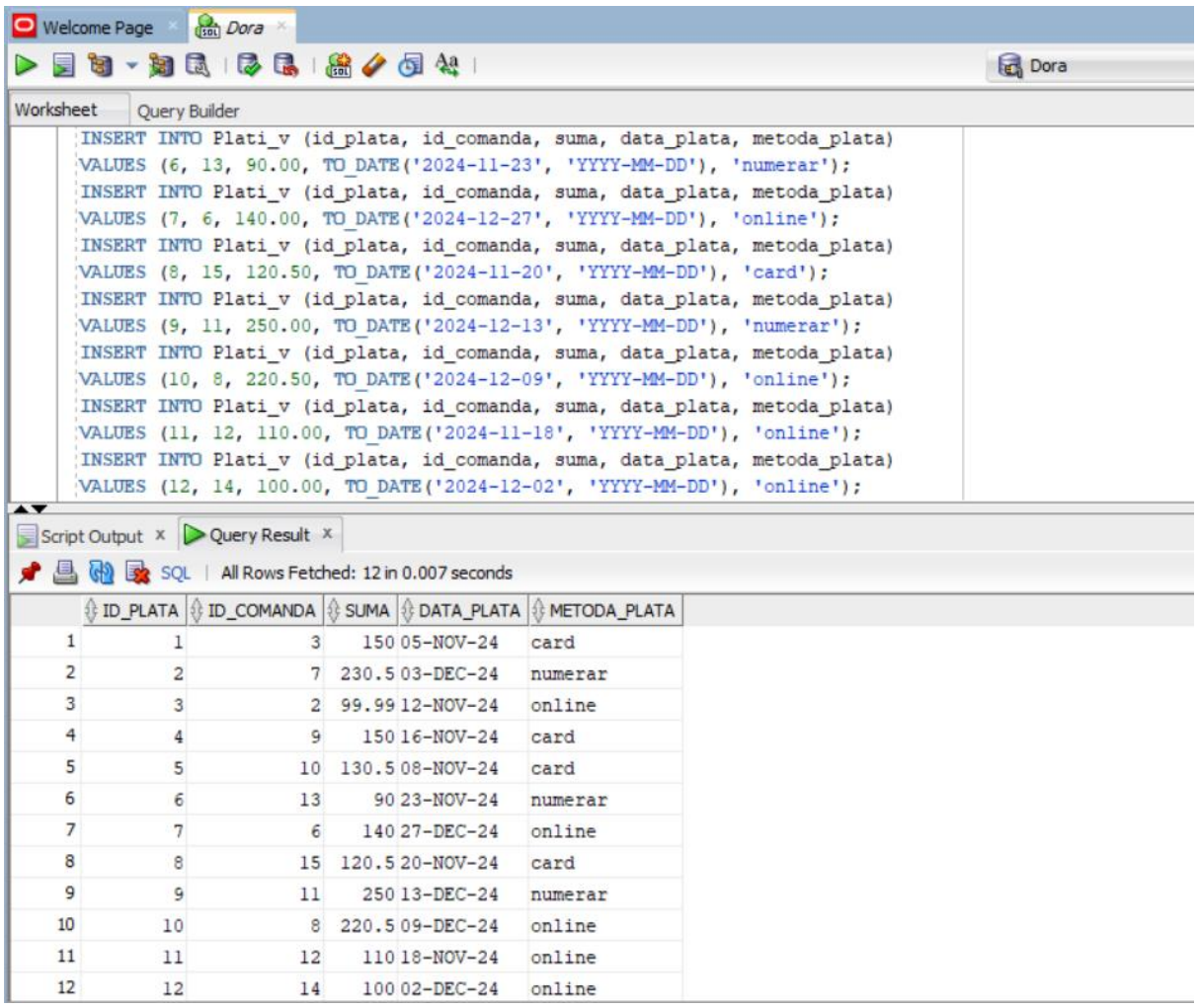
```
Name           Null?    Type
-------------- -------- ---------
ID_RECENZIE    NOT NULL NUMBER
ID_PRODUS               NUMBER
SCOR                    NUMBER(1)
DATA_RECENZIE           DATE
```

4. **Crearea tabelelor noi: Plati_v, Livrari_v, Recenzii_v**

```
CREATE TABLE Plati_v (
   id_plata NUMBER PRIMARY KEY,
   id_comanda NUMBER REFERENCES comenzi_v(id_comanda),
   suma NUMBER(10, 2) NOT NULL,
   data_plata DATE DEFAULT SYSDATE,
   metoda_plata VARCHAR2(50) -- de exemplu: card, numerar
);

INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (1, 3, 150.00, TO_DATE('2024-11-05', 'YYYY-MM-DD'), 'card');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (2, 7, 230.50, TO_DATE('2024-12-03', 'YYYY-MM-DD'), 'numerar');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (3, 2, 99.99, TO_DATE('2024-11-12', 'YYYY-MM-DD'), 'online');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (4, 9, 150.00, TO_DATE('2024-11-16', 'YYYY-MM-DD'), 'card');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (5, 10, 130.50, TO_DATE('2024-11-08', 'YYYY-MM-DD'), 'card');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (6, 13, 90.00, TO_DATE('2024-11-23', 'YYYY-MM-DD'), 'numerar');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (7, 6, 140.00, TO_DATE('2024-12-27', 'YYYY-MM-DD'), 'online');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
```

VALUES (8, 15, 120.50, TO_DATE('2024-11-20', 'YYYY-MM-DD'), 'card');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (9, 11, 250.00, TO_DATE('2024-12-13', 'YYYY-MM-DD'), 'numerar');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (10, 8, 220.50, TO_DATE('2024-12-09', 'YYYY-MM-DD'), 'online');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (11, 12, 110.00, TO_DATE('2024-11-18', 'YYYY-MM-DD'), 'online');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (12, 14, 100.00, TO_DATE('2024-12-02', 'YYYY-MM-DD'), 'online');

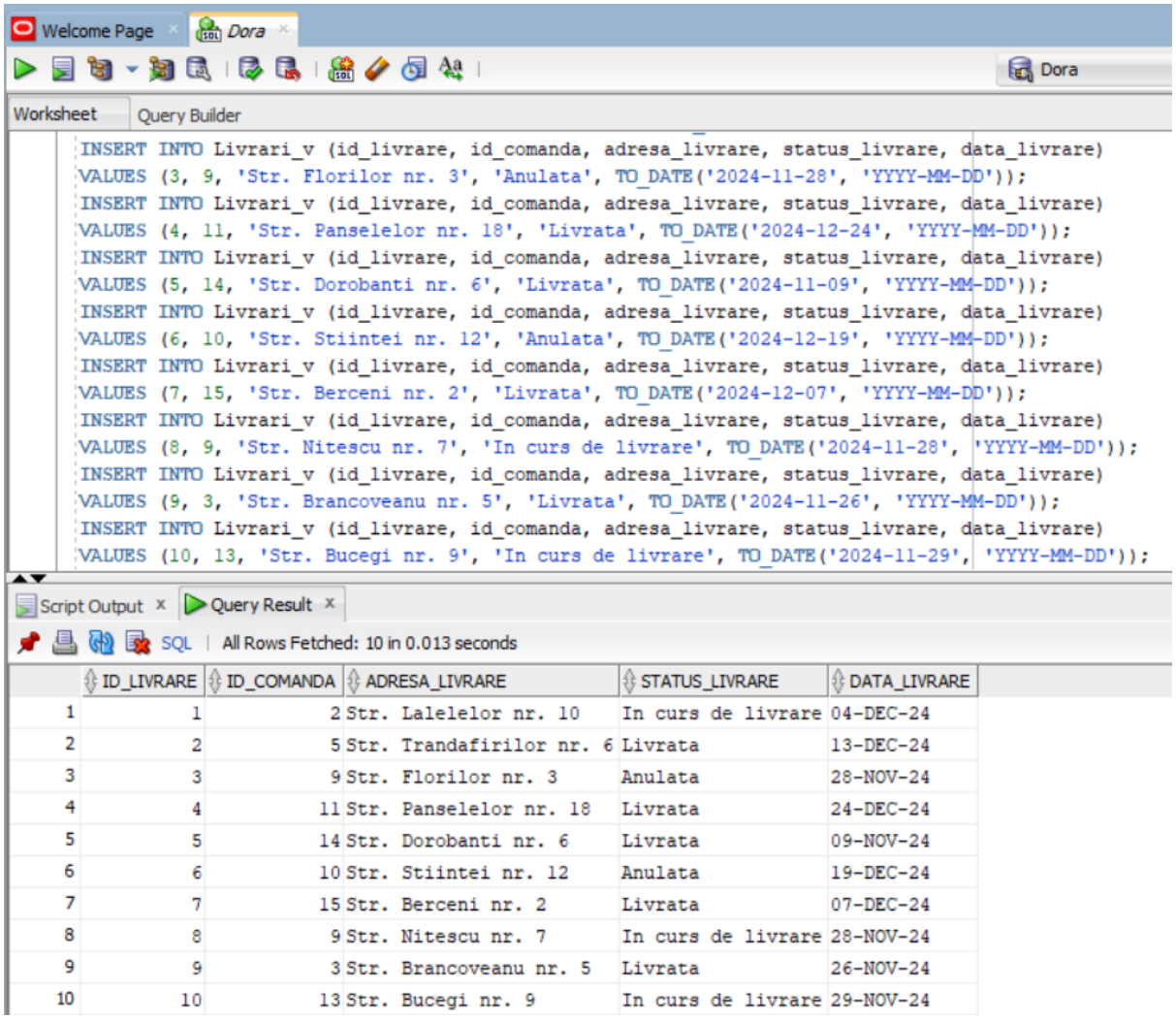



```
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (6, 13, 90.00, TO_DATE('2024-11-23', 'YYYY-MM-DD'), 'numerar');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (7, 6, 140.00, TO_DATE('2024-12-27', 'YYYY-MM-DD'), 'online');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (8, 15, 120.50, TO_DATE('2024-11-20', 'YYYY-MM-DD'), 'card');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (9, 11, 250.00, TO_DATE('2024-12-13', 'YYYY-MM-DD'), 'numerar');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (10, 8, 220.50, TO_DATE('2024-12-09', 'YYYY-MM-DD'), 'online');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (11, 12, 110.00, TO_DATE('2024-11-18', 'YYYY-MM-DD'), 'online');
INSERT INTO Plati_v (id_plata, id_comanda, suma, data_plata, metoda_plata)
VALUES (12, 14, 100.00, TO_DATE('2024-12-02', 'YYYY-MM-DD'), 'online');
```

	ID_PLATA	ID_COMANDA	SUMA	DATA_PLATA	METODA_PLATA
1	1	3	150	05-NOV-24	card
2	2	7	230.5	03-DEC-24	numerar
3	3	2	99.99	12-NOV-24	online
4	4	9	150	16-NOV-24	card
5	5	10	130.5	08-NOV-24	card
6	6	13	90	23-NOV-24	numerar
7	7	6	140	27-DEC-24	online
8	8	15	120.5	20-NOV-24	card
9	9	11	250	13-DEC-24	numerar
10	10	8	220.5	09-DEC-24	online
11	11	12	110	18-NOV-24	online
12	12	14	100	02-DEC-24	online

```
CREATE TABLE Livrari_v (
    id_livrare NUMBER PRIMARY KEY,
    id_comanda NUMBER REFERENCES comenzi_v(id_comanda),
    adresa_livrare VARCHAR2(200) NOT NULL,
    status_livrare VARCHAR2(50), -- de exemplu: in curs, livrata, anulata
    data_livrare DATE
);
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare,
data_livrare)
```

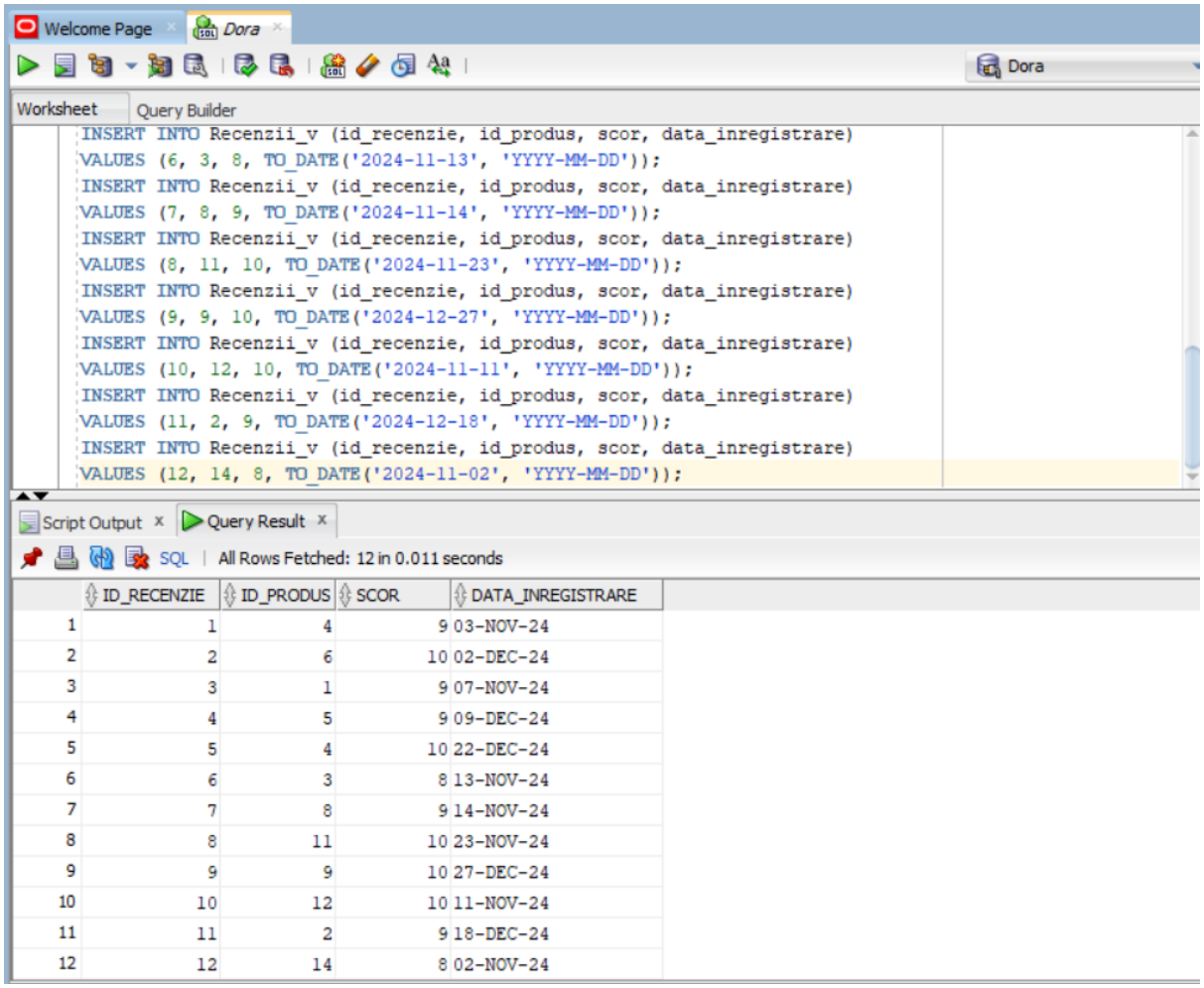
```
VALUES (1, 2, 'Str. Lalelelor nr. 10', 'In curs de livrare', TO_DATE('2024-12-04', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (2, 5, 'Str. Trandafirilor nr. 6', 'Livrata', TO_DATE('2024-12-13', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (3, 9, 'Str. Florilor nr. 3', 'Anulata', TO_DATE('2024-11-28', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (4, 11, 'Str. Panselelor nr. 18', 'Livrata', TO_DATE('2024-12-24', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (5, 14, 'Str. Dorobanti nr. 6', 'Livrata', TO_DATE('2024-11-09', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (6, 10, 'Str. Stiintei nr. 12', 'Anulata', TO_DATE('2024-12-19', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (7, 15, 'Str. Berceni nr. 2', 'Livrata', TO_DATE('2024-12-07', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (8, 9, 'Str. Nitescu nr. 7', 'In curs de livrare', TO_DATE('2024-11-28', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (9, 3, 'Str. Brancoveanu nr. 5', 'Livrata', TO_DATE('2024-11-26', 'YYYY-MM-DD'));
INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
VALUES (10, 13, 'Str. Bucegi nr. 9', 'In curs de livrare', TO_DATE('2024-11-29', 'YYYY-MM-DD'));
```




```
CREATE TABLE Recenzii_v (
    id_recenzie NUMBER PRIMARY KEY,
    id_produs NUMBER REFERENCES produse_v(id_produs),
    scor NUMBER(2) CHECK (scor BETWEEN 1 AND 10),
    data_inregistrare DATE DEFAULT SYSDATE
);

INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (1, 4, 9, TO_DATE('2024-11-03', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (2, 6, 10, TO_DATE('2024-12-02', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (3, 1, 9, TO_DATE('2024-11-07', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (4, 5, 9, TO_DATE('2024-12-09', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (5, 4, 10, TO_DATE('2024-12-22', 'YYYY-MM-DD'));
```

```
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (6, 3, 8, TO_DATE('2024-11-13', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (7, 8, 9, TO_DATE('2024-11-14', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (8, 11, 10, TO_DATE('2024-11-23', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (9, 9, 10, TO_DATE('2024-12-27', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (10, 12, 10, TO_DATE('2024-11-11', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (11, 2, 9, TO_DATE('2024-12-18', 'YYYY-MM-DD'));
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (12, 14, 8, TO_DATE('2024-11-02', 'YYYY-MM-DD'));
```



	ID_RECENZIE	ID_PRODUS	SCOR	DATA_INREGISTRARE
1	1	4	9	03-NOV-24
2	2	6	10	02-DEC-24
3	3	1	9	07-NOV-24
4	4	5	9	09-DEC-24
5	5	4	10	22-DEC-24
6	6	3	8	13-NOV-24
7	7	8	9	14-NOV-24
8	8	11	10	23-NOV-24
9	9	9	10	27-DEC-24
10	10	12	10	11-NOV-24
11	11	2	9	18-DEC-24
12	12	14	8	02-NOV-24

SEMINAR 1: Introducere în PL/SQL. Interacțiunea cu serverul Oracle prin comenzi SQL

1. **Comenzi LDD**

- **Comanda CREATE**

```
SET SERVEROUTPUT ON
DECLARE
v_id_manager angajati_v.id_manager%TYPE;
BEGIN
   select id_manager into v_id_manager from angajati_v where prenume='Carina';
   execute immediate 'create table angajati_sup_v
   as select * from angajati_v where id_manager='||v_id_manager;
   dbms_output.put_line('Am creat o tabela care contine doar angajatii care o au ca manager pe Carina.');
END;
/
```

```
Am creat o tabela care contine doar angajatii care o au ca manager pe Carina.


PL/SQL procedure successfully completed.
```

```
select * from angajati_sup_v;
```

Script Output | Query Result | SQL | All Rows Fetched: 9 in 0.041 seconds

	ID_ANGAJAT	PRENUME	NUME	TELEFON	EMAIL	DATA_ANGAJARE	SALARIUL	ID_FUNCTIE	ID_MANAGER
1	1	Ana	Marinescu	0735264387	marinana65@gmail.com	01-FEB-19	6050	1	8
2	2	Costel	David	0747231645	costeldav4@gmail.com	24-JUN-19	5940	1	8
3	3	Crina	Popescu	0754087648	crinapop3@yahoo.com	13-MAR-23	4620	2	8
4	4	Andra	Georgescu	0734123627	anageorg5@gmail.com	02-JUN-23	4730	2	8
5	5	Sorina	Lungu	0734175329	sorina32@yahoo.com	08-APR-23	4620	2	8
6	6	Cristian	Vasilescu	0782598756	cristian6@yahoo.com	19-NOV-22	3700	3	8
7	7	Codrin	Stancu	0798231593	codrin3@gmail.com	15-OCT-22	3800	3	8
8	8	Carina	Samoila	0752839654	carinasam9@gmail.com	30-APR-21	2700	4	8
9	9	Ioana	Marin	0783175324	ioanam8@yahoo.com	29-APR-22	2600	5	8

- **Comanda ALTER**

```
BEGIN
   execute immediate 'alter table angajati_sup_v add bonus NUMBER(10,2)';
   dbms_output.put_line('Am adaugat o coloana de bonusuri pentru angajati.');
END;
/
```

```
Am adaugat o coloana de bonusuri pentru angajati.


PL/SQL procedure successfully completed.
```

```
select * from angajati_sup_v;
```

Script Output | Query Result

SQL | All Rows Fetched: 9 in 0.022 seconds

	_ANGAJAT	PRENUME	NUME	TELEFON	EMAIL	DATA_ANGAJARE	SALARIUL	ID_FUNCTIE	ID_MANAGER	BONUS
1	1	Ana	Marinescu	0735264387	marinana65@gmail.com	01-FEB-19	6050	1	8	(null)
2	2	Costel	David	0747231645	costeldav4@gmail.com	24-JUN-19	5940	1	8	(null)
3	3	Crina	Popescu	0754087648	crinapop3@yahoo.com	13-MAR-23	4620	2	8	(null)
4	4	Andra	Georgescu	0734123627	anageorg5@gmail.com	02-JUN-23	4730	2	8	(null)
5	5	Sorina	Lungu	0734175329	sorina32@yahoo.com	08-APR-23	4620	2	8	(null)
6	6	Cristian	Vasilescu	0782598756	cristian6@yahoo.com	19-NOV-22	3700	3	8	(null)
7	7	Codrin	Stancu	0798231593	codrin3@gmail.com	15-OCT-22	3800	3	8	(null)
8	8	Carina	Samoila	0752839654	carinasam9@gmail.com	30-APR-21	2700	4	8	(null)
9	9	Ioana	Marin	0783175324	ioanam8@yahoo.com	29-APR-22	2600	5	8	(null)

- **Comanda DELETE**

```
BEGIN
   execute immediate 'drop table angajati_sup_v';
   dbms_output.put_line('Am sters tabela angajati_sup_v');
END;
/
```

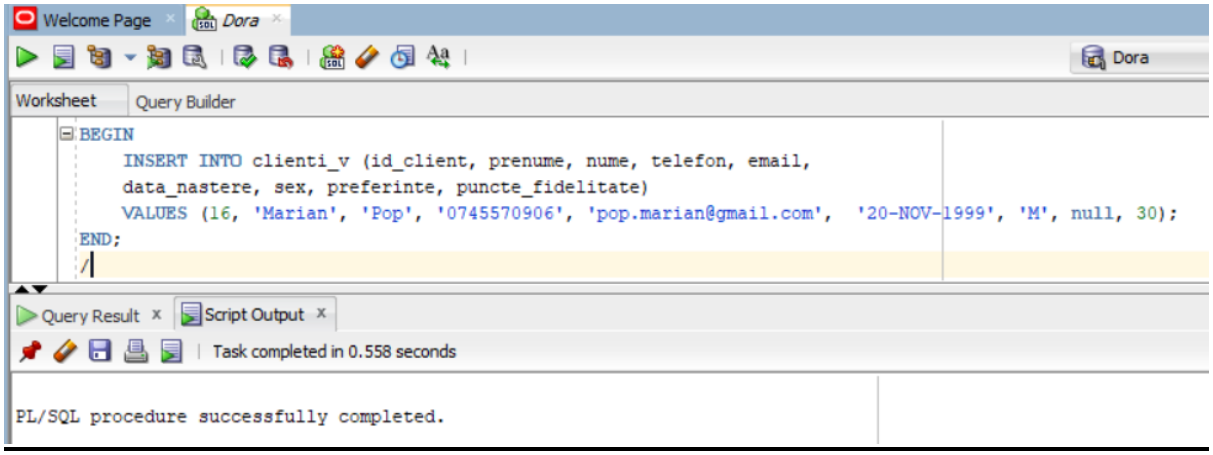
```
Am sters tabela angajati_sup_v


PL/SQL procedure successfully completed.
```

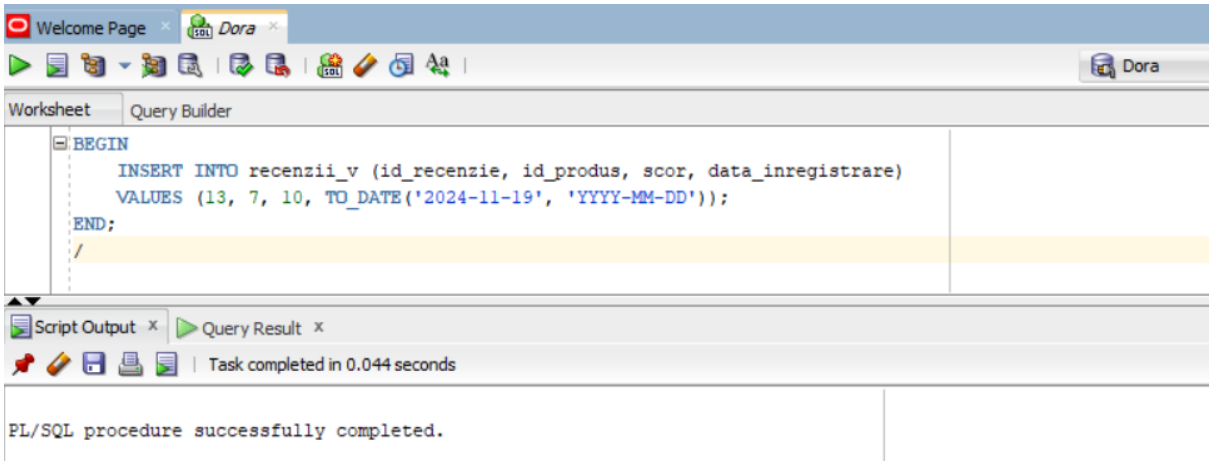
2. **Manipularea datelor** in PL/SQL prin instrucţiunile **LMD** (INSERT, UPDATE, DELETE)

- **Comanda INSERT**

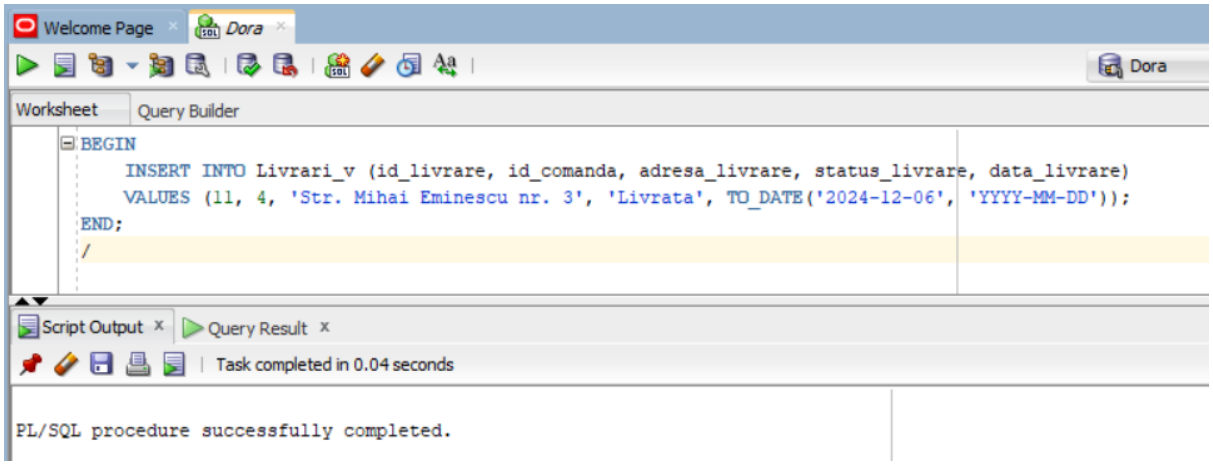
```
BEGIN
        INSERT INTO clienti_v (id_client, prenume, nume, telefon, email,
        data_nastere, sex, preferinte, puncte_fidelitate)
        VALUES (16, 'Marian', 'Pop', '0745570906', 'pop.marian@gmail.com',  '20-NOV-
        1999', 'M', null, 30);
END;
/
```




```
BEGIN
   INSERT INTO recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
   VALUES (13, 7, 10, TO_DATE('2024-11-19', 'YYYY-MM-DD'));
END;
/
```




```
BEGIN
   INSERT INTO Livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare,
data_livrare)
   VALUES (11, 4, 'Str. Mihai Eminescu nr. 3', 'Livrata', TO_DATE('2024-12-06', 'YYYY-
MM-DD'));
END;
/
```



- **<u>Comanda UPDATE</u>**

```
SET SERVEROUTPUT ON
DECLARE
v_pret number:=20;
BEGIN
   update rand_comenzi_v
   set pret=pret++v_pret;
END;
/
```

```
Am crescut prețul produselor cu 20 de lei.


PL/SQL procedure successfully completed.
```

```
BEGIN
   update produse_v
   set personalizabil='DA'
   where initcap(categorie)='Torturi';
   dbms_output.put_line('Am facut toate torturile personalizabile.');
END;
/
```

```
Am facut toate torturile personalizabile.


PL/SQL procedure successfully completed.
```

```
BEGIN
   update livrari_v
   set status_livrare='Livrata'
   where id_livrare=8;
   dbms_output.put_line('Am modificat livrarea cu id-ul 8 ca livrata.');
END;
```

```
Am modificat livrarea cu id-ul 8 ca livrata.


PL/SQL procedure successfully completed.
```

- **Comanda DELETE**

```
BEGIN
   delete from recenzii_v where scor<9;
   rollback;
   dbms_output.put_line('Stergem recenziile cu un scor mai mic de 9, apoi anulam
modificarea.');
END;
```

```
Stergem recenziile cu un scor mai mic de 9, apoi anulam modificarea.


PL/SQL procedure successfully completed.
```

```
BEGIN
   delete from livrari_v where initcap(status_livrare)='Anulata';
   dbms_output.put_line('Stergem livrarile anulate.');
END;
```

```
Stergem livrarile anulate.


PL/SQL procedure successfully completed.
```

```
select * from livrari_v;
```

Script Output | Query Result

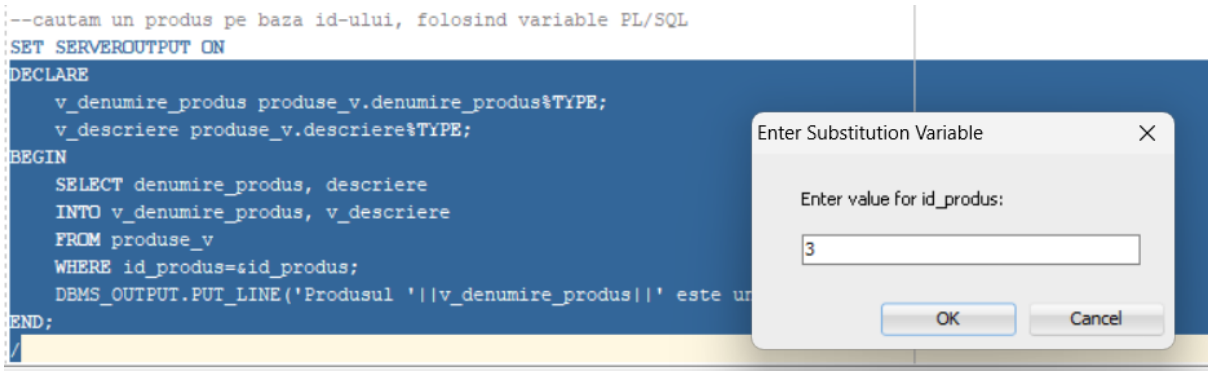
SQL | All Rows Fetched: 8 in 0.004 seconds

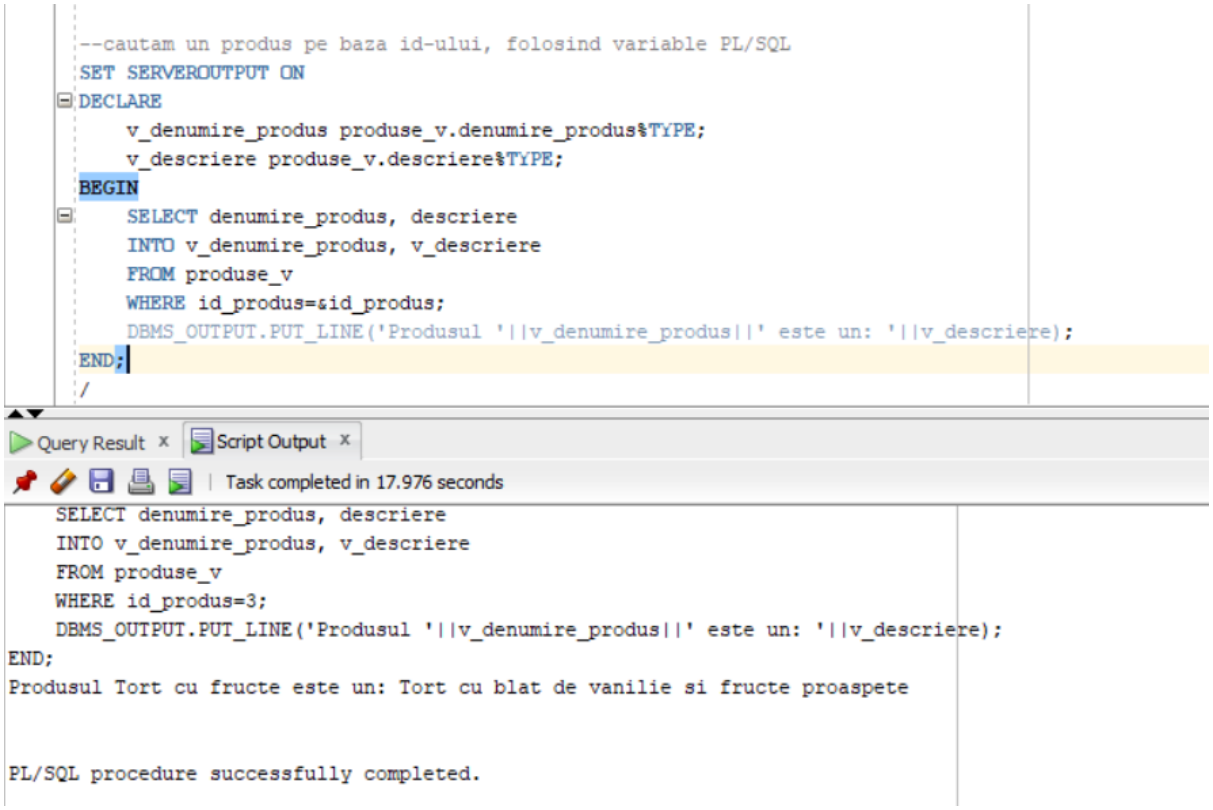
	ID_LIVRARE	ID_COMANDA	ADRESA_LIVRARE	STATUS_LIVRARE	DATA_LIVRARE
1	1	2	Str. Lalelelor nr. 10	In curs de livrare	04-DEC-24
2	2	5	Str. Trandafirilor nr. 6	Livrata	13-DEC-24
3	4	11	Str. Panselelor nr. 18	Livrata	24-DEC-24
4	5	14	Str. Dorobanti nr. 6	Livrata	09-NOV-24
5	7	15	Str. Berceni nr. 2	Livrata	07-DEC-24
6	8	9	Str. Nitescu nr. 7	In curs de livrare	28-NOV-24
7	9	3	Str. Brancoveanu nr. 5	Livrata	26-NOV-24
8	10	13	Str. Bucegi nr. 9	In curs de livrare	29-NOV-24

SEMINAR 2: Variabilele PL/SQL

- Căutăm un produs al cofetăriei folosind o variabilă pentru id.

```
SET SERVEROUTPUT ON
DECLARE
  v_denumire_produs produse_v.denumire_produs%TYPE;
  v_descriere produse_v.descriere%TYPE;
BEGIN
  SELECT denumire_produs,descriere
    INTO v_denumire_produs,v_descriere
    FROM produse_v
  WHERE id_produs = &id_produs;  -- la execuție vei fi întrebat de id_produs
  DBMS_OUTPUT.PUT_LINE('Produsul ' || v_denumire_produs || ' este un: ' || v_descriere);
END;
/
```





```
--cautam un produs pe baza id-ului, folosind variable PL/SQL
SET SERVEROUTPUT ON
DECLARE
    v_denumire_produs produse_v.denumire_produs%TYPE;
    v_descriere produse_v.descriere%TYPE;
BEGIN
    SELECT denumire_produs, descriere
    INTO v_denumire_produs, v_descriere
    FROM produse_v
    WHERE id_produs=&id_produs;
    DBMS_OUTPUT.PUT_LINE('Produsul '||v_denumire_produs||' este un: '||v_descriere);
END;
/
```

Query Result | Script Output

Task completed in 17.976 seconds

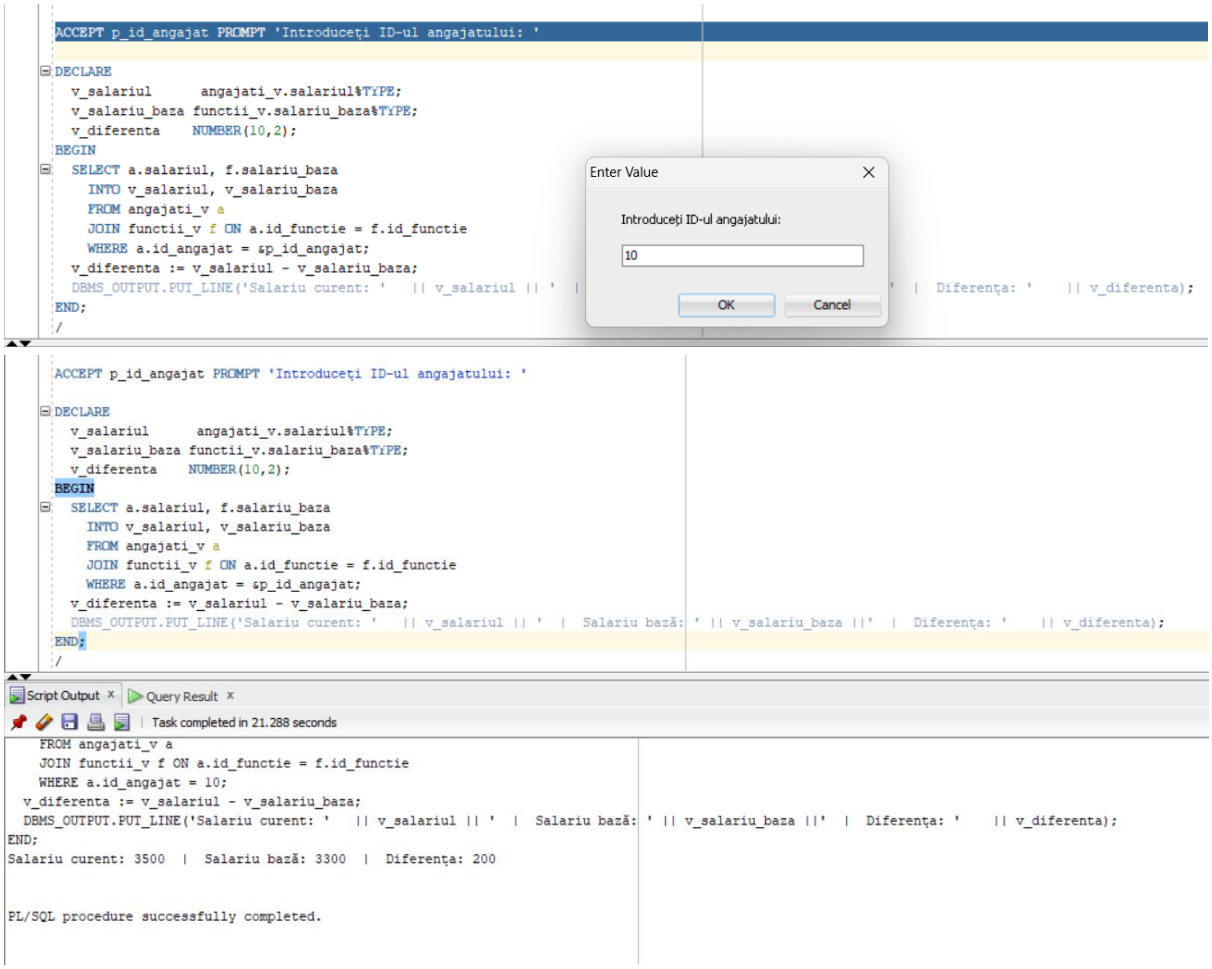
```
    SELECT denumire_produs, descriere
    INTO v_denumire_produs, v_descriere
    FROM produse_v
    WHERE id_produs=3;
    DBMS_OUTPUT.PUT_LINE('Produsul '||v_denumire_produs||' este un: '||v_descriere);
END;
Produsul Tort cu fructe este un: Tort cu blat de vanilie si fructe proaspete


PL/SQL procedure successfully completed.
```

- Căutăm un angajat după id și calculăm diferența dintre salariul curent și cel de bază.

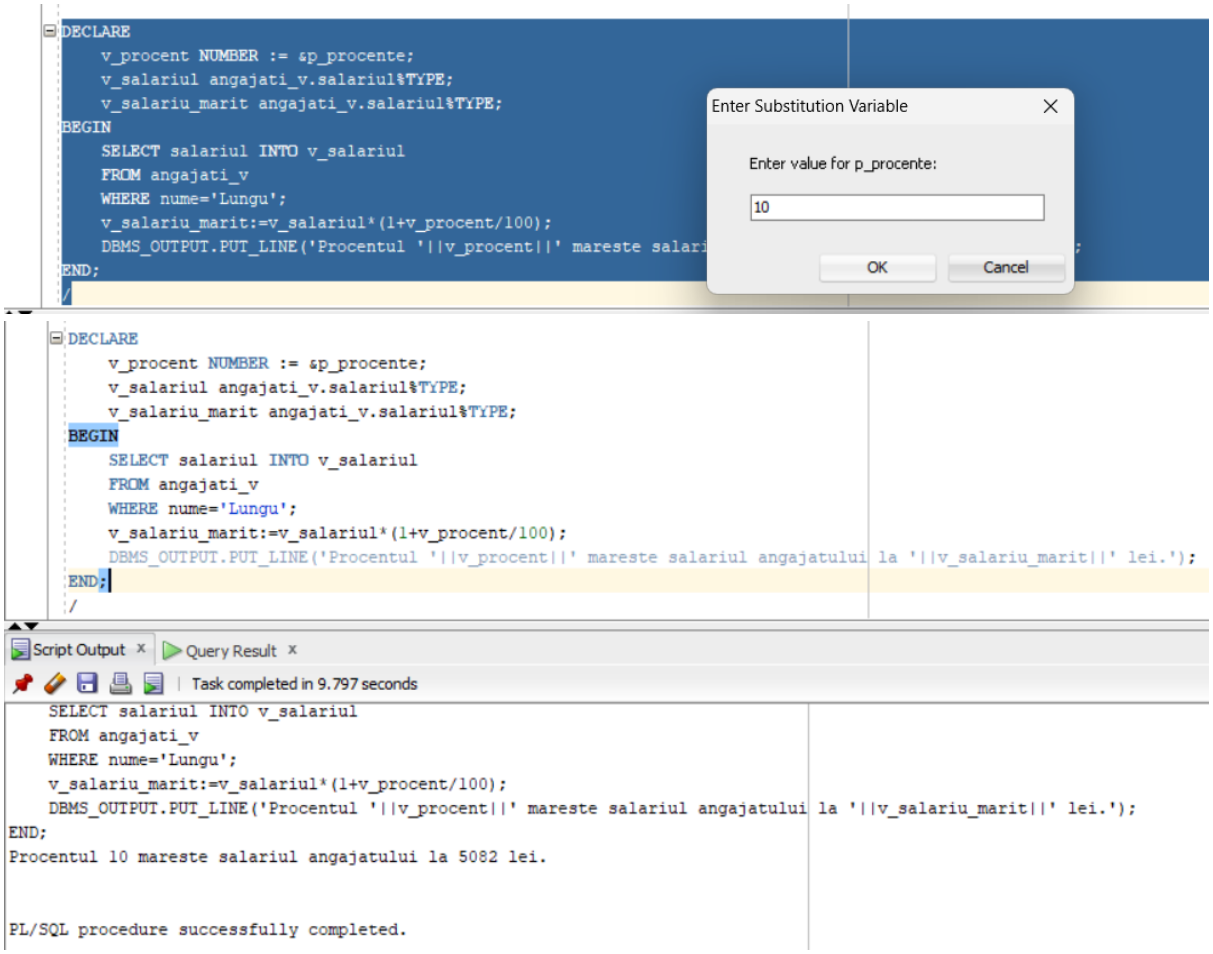
```
ACCEPT p_id_angajat PROMPT 'Introduceți ID-ul angajatului: '

DECLARE
  v_salariul      angajati_v.salariul%TYPE;
  v_salariu_baza functii_v.salariu_baza%TYPE;
  v_diferenta    NUMBER(10,2);
BEGIN
  SELECT a.salariul, f.salariu_baza
    INTO v_salariul, v_salariu_baza
    FROM angajati_v a
    JOIN functii_v f ON a.id_functie = f.id_functie
    WHERE a.id_angajat = &p_id_angajat;
  v_diferenta := v_salariul - v_salariu_baza;
  DBMS_OUTPUT.PUT_LINE('Salariu curent: '   || v_salariul || ' | Salariu bază: ' ||
v_salariu_baza ||' | Diferența: '    || v_diferenta);
END;
/
```



- Calculăm salariul angajatului Lungu la potențiala mărire cu 10%.

```
DECLARE
   v_procent NUMBER := &p_procente;
   v_salariul angajati_v.salariul%TYPE;
   v_salariu_marit angajati_v.salariul%TYPE;
BEGIN
   SELECT salariul INTO v_salariul
   FROM angajati_v
   WHERE nume='Lungu';
   v_salariu_marit:=v_salariul*(1+v_procent/100);
   DBMS_OUTPUT.PUT_LINE('Procentul '||v_procent||' mareste salariul angajatului la '||v_salariu_marit||' lei.');
END;
/
```



SEMINAR 3: Structuri fundamentale de programare

- **Exemplu IF…THEN…ELSIF…END IF**

ACCEPT p_id_recenzie PROMPT 'Introduceți ID-ul recenziei: '

```
DECLARE
  v_scor recenzii_v.scor%TYPE;
BEGIN
   SELECT scor
   INTO v_scor
   FROM recenzii_v
   WHERE id_recenzie = &p_id_recenzie;
   IF v_scor >= 8 THEN
      DBMS_OUTPUT.PUT_LINE('Recenzie pozitiva (scor=' || v_scor || ')');
   ELSIF v_scor >= 5 THEN
      DBMS_OUTPUT.PUT_LINE('Recenzie neutra (scor='  || v_scor || ')');
   ELSE
```

```
      DBMS_OUTPUT.PUT_LINE('Recenzie negativa (scor=' || v_scor || ')');
  END IF;
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('Nu exista recenzie cu ID-ul dat.');
END;
/
```

```
ACCEPT p_id_recenzie PROMPT 'Introduceți ID-ul recenziei: '

DECLARE
  v_scor recenzii_v.scor%TYPE;
BEGIN
    SELECT scor
    INTO v_scor
    FROM recenzii_v
    WHERE id_recenzie = &p_id_recenzie;
    IF v_scor >= 8 THEN
        DBMS_OUTPUT.PUT_LINE('Recenzie pozitiva (scor=' || v_scor || ')');
    ELSIF v_scor >= 5 THEN
        DBMS_OUTPUT.PUT_LINE('Recenzie neutra (scor='  || v_scor || ')');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Recenzie negativa (scor=' || v_scor || ')');
  END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Nu exista recenzie cu ID-ul dat.');
END;
/
```

Script Output | Query Result

Task completed in 4.323 seconds

```
        DBMS_OUTPUT.PUT_LINE('Nu exista recenzie cu ID-ul dat.');
END;
Recenzie pozitiva (scor=8)


PL/SQL procedure successfully completed.
```

- **Exemplu CASE … WHEN … THEN … END CASE**

```
ACCEPT p_id_produs PROMPT 'Introduceți ID-ul produsului: '
DECLARE
  v_categorie produse_v.categorie%TYPE;
  v_mesaj VARCHAR2(200);
BEGIN
   SELECT categorie
   INTO v_categorie
```

```
    FROM produse_v
    WHERE id_produs = &p_id_produs;
    v_mesaj := CASE
        WHEN v_categorie = 'Torturi'    THEN 'Acesta este un tort delicios!'
        WHEN v_categorie = 'Prajituri'  THEN 'Aceasta este o prajitura aromata!'
        ELSE 'Categorie necunoscută: ' || v_categorie
    END;
    DBMS_OUTPUT.PUT_LINE(v_mesaj);
END;
/
```

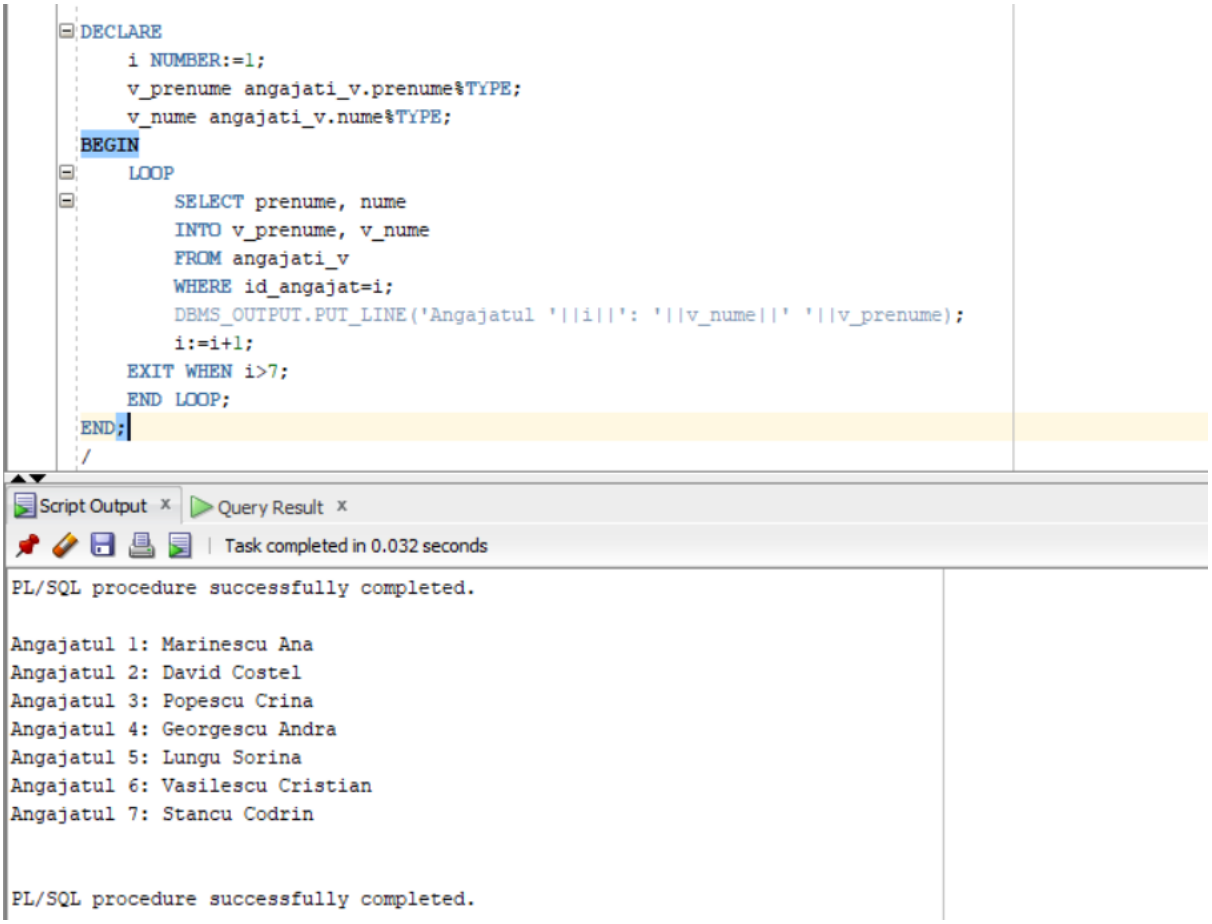



```
Aceasta este o prajitura aromata!


PL/SQL procedure successfully completed.
```

- **Exemplu LOOP … EXIT WHEN … END LOOP**

```
DECLARE
    i NUMBER:=1;
    v_prenume angajati_v.prenume%TYPE;
    v_nume angajati_v.nume%TYPE;
BEGIN
    LOOP
        SELECT prenume, nume
        INTO v_prenume, v_nume
        FROM angajati_v
        WHERE id_angajat=i;
        DBMS_OUTPUT.PUT_LINE('Angajatul '||i||': '||v_nume||' '||v_prenume);
        i:=i+1;
    EXIT WHEN i>7;
    END LOOP;
END;
```



```
DECLARE
    i NUMBER:=1;
    v_prenume angajati_v.prenume%TYPE;
    v_nume angajati_v.nume%TYPE;
BEGIN
    LOOP
        SELECT prenume, nume
        INTO v_prenume, v_nume
        FROM angajati_v
        WHERE id_angajat=i;
        DBMS_OUTPUT.PUT_LINE('Angajatul '||i||': '||v_nume||' '||v_prenume);
        i:=i+1;
    EXIT WHEN i>7;
    END LOOP;
END;
/
```

Script Output | Query Result

Task completed in 0.032 seconds

```
PL/SQL procedure successfully completed.

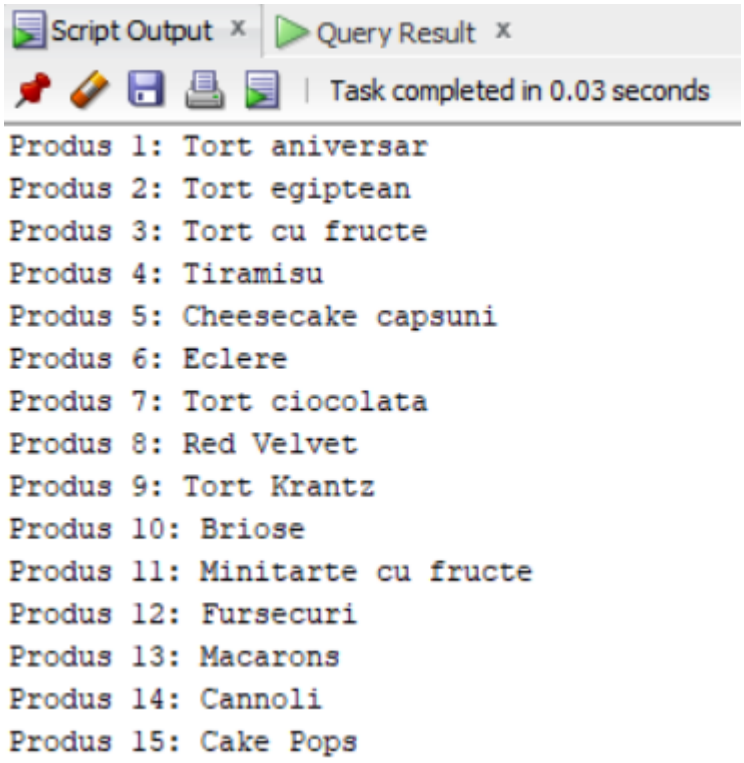
Angajatul 1: Marinescu Ana
Angajatul 2: David Costel
Angajatul 3: Popescu Crina
Angajatul 4: Georgescu Andra
Angajatul 5: Lungu Sorina
Angajatul 6: Vasilescu Cristian
Angajatul 7: Stancu Codrin


PL/SQL procedure successfully completed.
```

- **Exemplu WHILE … LOOP … END LOOP**

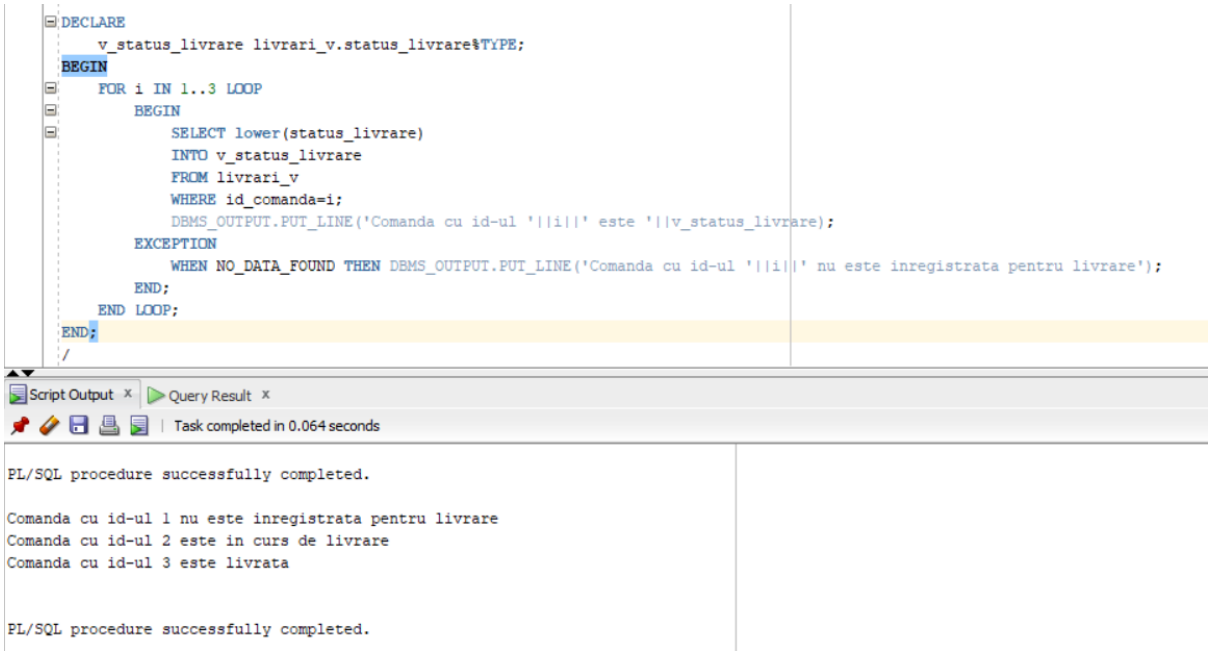
```
DECLARE
   v_max NUMBER;
   i NUMBER := 1;
   v_nume produse_v.denumire_produs%TYPE;
BEGIN
   SELECT COUNT(*) INTO v_max FROM produse_v;
   WHILE i <= v_max LOOP
   BEGIN
      SELECT denumire_produs
         INTO v_nume
         FROM produse_v
      WHERE id_produs = i;
      DBMS_OUTPUT.PUT_LINE('Produs ' || i || ': ' || v_nume);
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         NULL;
   END;
   i := i + 1;
```

```
  END LOOP;
END;
/
```



- **Exemplu FOR … LOOP … END LOOP**

```
DECLARE
   v_status_livrare livrari_v.status_livrare%TYPE;
BEGIN
   FOR i IN 1..3 LOOP
      BEGIN
         SELECT lower(status_livrare)
         INTO v_status_livrare
         FROM livrari_v
         WHERE id_comanda=i;
         DBMS_OUTPUT.PUT_LINE('Comanda cu id-ul '||i||' este '||v_status_livrare);
      EXCEPTION
         WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Comanda cu id-
ul '||i||' nu este inregistrata pentru livrare');
      END;
   END LOOP;
END;
/
```



```
DECLARE
    v_status_livrare livrari_v.status_livrare%TYPE;
BEGIN
    FOR i IN 1..3 LOOP
        BEGIN
            SELECT lower(status_livrare)
            INTO v_status_livrare
            FROM livrari_v
            WHERE id_comanda=i;
            DBMS_OUTPUT.PUT_LINE('Comanda cu id-ul '||i||' este '||v_status_livrare);
        EXCEPTION
            WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Comanda cu id-ul '||i||' nu este inregistrata pentru livrare');
        END;
    END LOOP;
END;
/
```

Script Output | Query Result | Task completed in 0.064 seconds

```
PL/SQL procedure successfully completed.

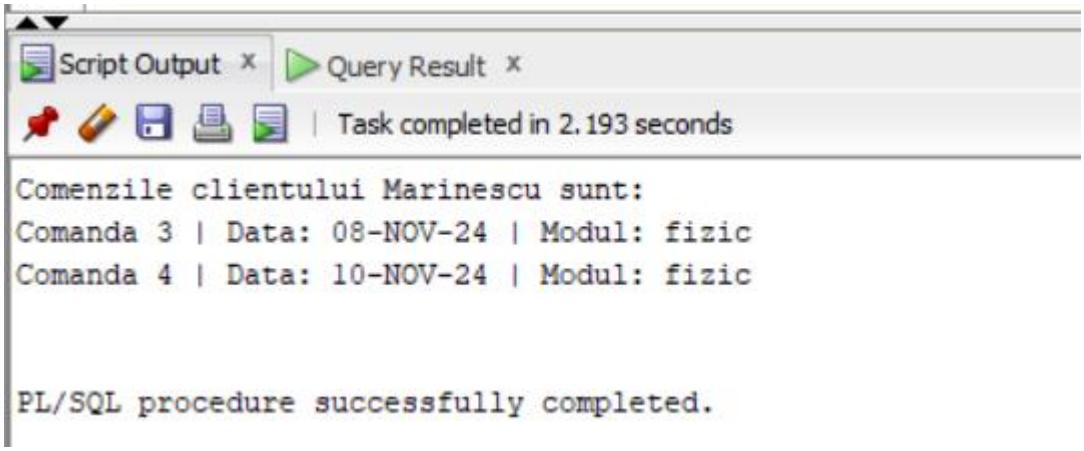
Comanda cu id-ul 1 nu este inregistrata pentru livrare
Comanda cu id-ul 2 este in curs de livrare
Comanda cu id-ul 3 este livrata


PL/SQL procedure successfully completed.
```

SEMINAR 4 ȘI 5: Cursori

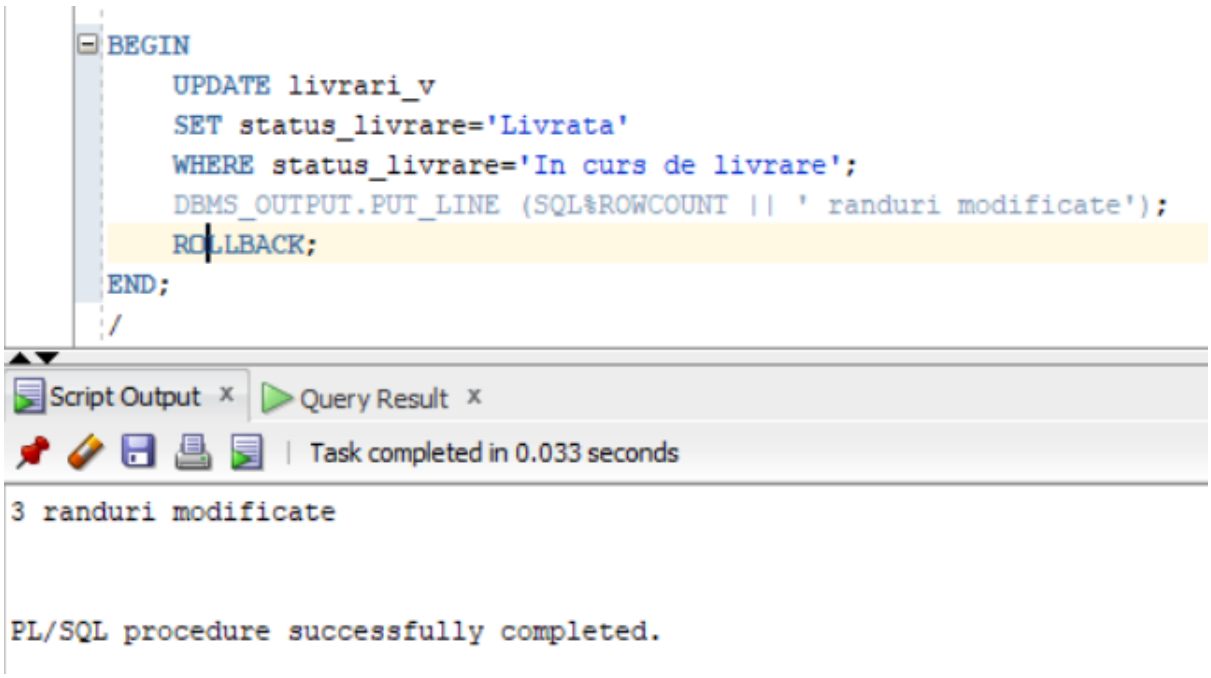
- Căutăm comenzile făcute de un client folosind id-ul său:

```
DECLARE
   CURSOR c_comenzi (p_id_client CLIENTI_v.id_client%TYPE) IS
   SELECT id_comanda, data, modalitate
   FROM comenzi_v
   WHERE id_client = p_id_client;
   v_comenzi_record c_comenzi%ROWTYPE;
   v_id_client NUMBER:=&v_id_client;
   v_nume clienti_v.nume%TYPE;
BEGIN
   SELECT nume INTO v_nume FROM clienti_v WHERE id_client=v_id_client;
   OPEN c_comenzi(v_id_client);
   DBMS_OUTPUT.PUT_LINE('Comenzile clientului '||v_nume||' sunt:');
   LOOP
      FETCH c_comenzi INTO v_comenzi_record;
      EXIT WHEN c_comenzi%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Comanda '||v_comenzi_record.id_comanda||' | Data: '||
v_comenzi_record.data||' | Modul: '||v_comenzi_record.modalitate);
   END LOOP;
   CLOSE c_comenzi;
END;
/
```



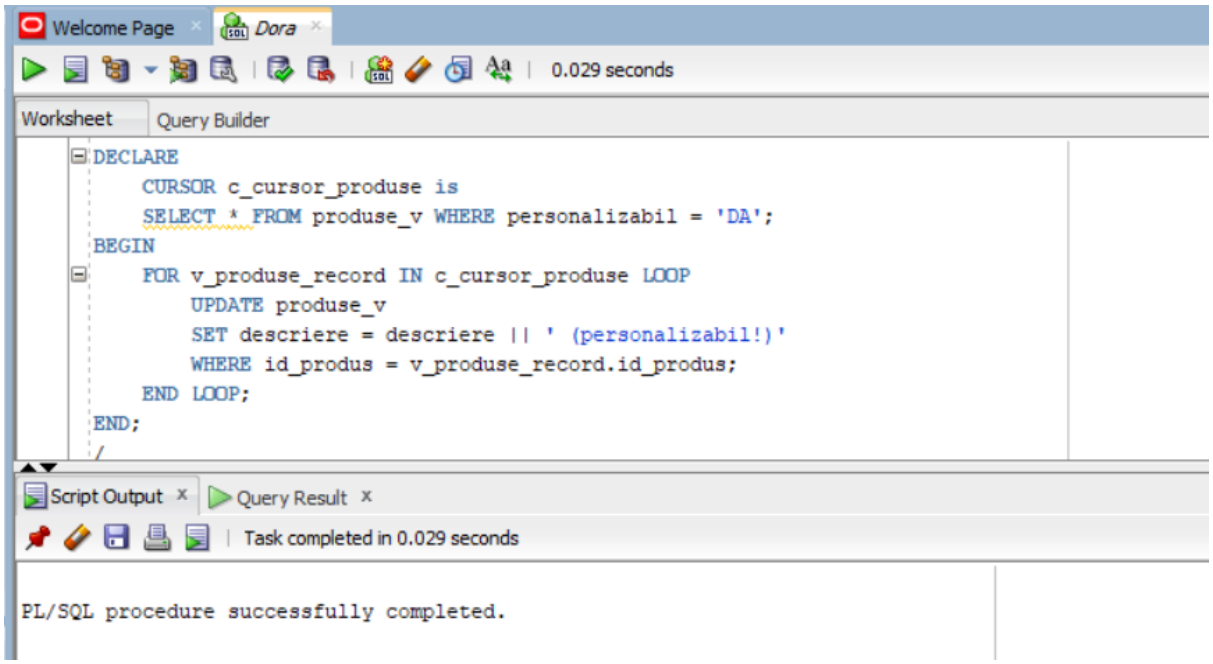
- Modificăm livrările “în curs de livrare” ca fiind deja livrate și numărăm câte sunt folosind SQL%ROWCOUNT

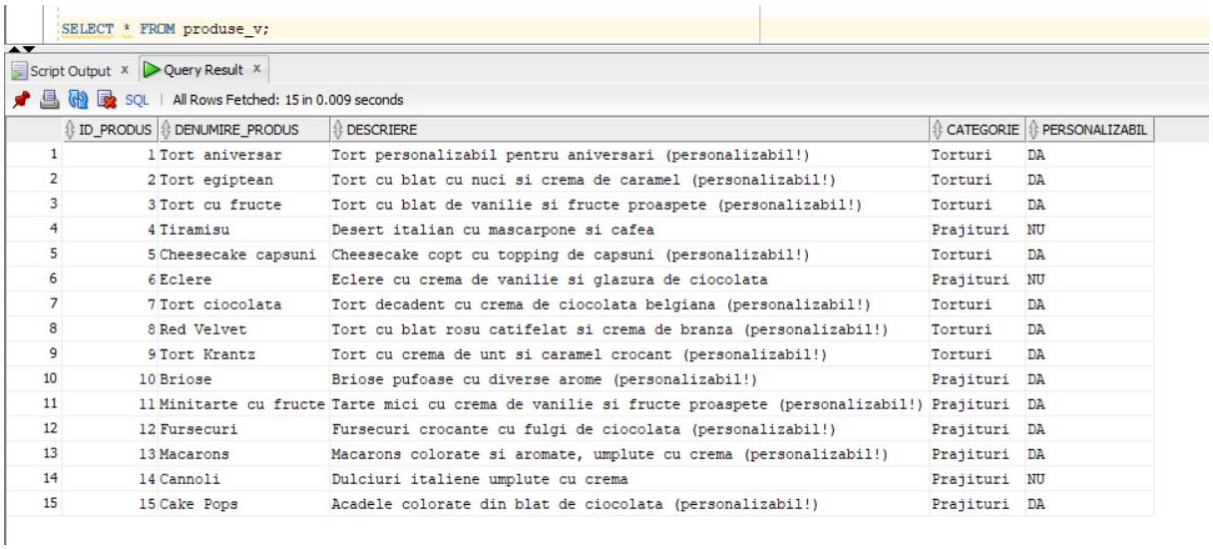
```
BEGIN
   UPDATE livrari_v
   SET status_livrare='Livrata'
   WHERE status_livrare='In curs de livrare';
   DBMS_OUTPUT.PUT_LINE (SQL%ROWCOUNT || ' randuri modificate');
   ROLLBACK;
END;
/
```



- Modificăm descrierea produselor personalizabile folosind un cursor explicit:

```
DECLARE
   CURSOR c_cursor_produse is
   SELECT * FROM produse_v WHERE personalizabil = 'DA';
BEGIN
   FOR v_produse_record IN c_cursor_produse LOOP
      UPDATE produse_v
      SET descriere = descriere || ' (personalizabil!)'
      WHERE id_produs = v_produse_record.id_produs;
   END LOOP;
END;
/
```

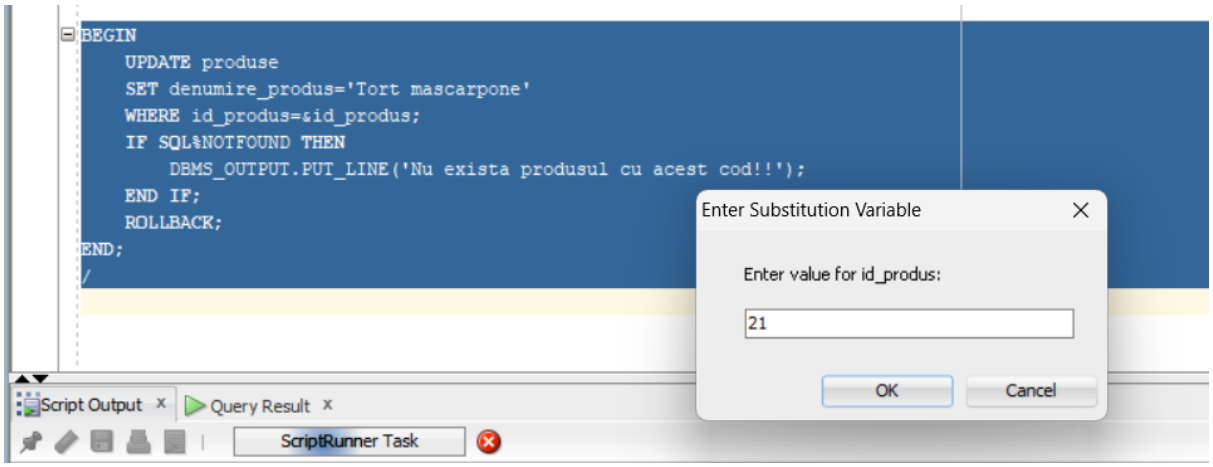




	ID_PRODUS	DENUMIRE_PRODUS	DESCRIERE	CATEGORIE	PERSONALIZABIL
1	1	Tort aniversar	Tort personalizabil pentru aniversari (personalizabil!)	Torturi	DA
2	2	Tort egiptean	Tort cu blat cu nuci si crema de caramel (personalizabil!)	Torturi	DA
3	3	Tort cu fructe	Tort cu blat de vanilie si fructe proaspete (personalizabil!)	Torturi	DA
4	4	Tiramisu	Desert italian cu mascarpone si cafea	Prajituri	NU
5	5	Cheesecake capsuni	Cheesecake copt cu topping de capsuni (personalizabil!)	Torturi	DA
6	6	Eclere	Eclere cu crema de vanilie si glazura de ciocolata	Prajituri	NU
7	7	Tort ciocolata	Tort decadent cu crema de ciocolata belgiana (personalizabil!)	Torturi	DA
8	8	Red Velvet	Tort cu blat rosu catifelat si crema de branza (personalizabil!)	Torturi	DA
9	9	Tort Krantz	Tort cu crema de unt si caramel crocant (personalizabil!)	Torturi	DA
10	10	Briose	Briose pufoase cu diverse arome (personalizabil!)	Prajituri	DA
11	11	Minitarte cu fructe	Tarte mici cu crema de vanilie si fructe proaspete (personalizabil!)	Prajituri	DA
12	12	Fursecuri	Fursecuri crocante cu fulgi de ciocolata (personalizabil!)	Prajituri	DA
13	13	Macarons	Macarons colorate si aromate, umplute cu crema (personalizabil!)	Prajituri	DA
14	14	Cannoli	Dulciuri italiene umplute cu crema	Prajituri	NU
15	15	Cake Pops	Acadele colorate din blat de ciocolata (personalizabil!)	Prajituri	DA

- Testăm dacă putem modifica numele unui produs folosind SQL%NOTFOUND:

```
BEGIN
   UPDATE produse
   SET denumire_produs='Tort mascarpone'
   WHERE id_produs=&id_produs;
   IF SQL%NOTFOUND THEN
      DBMS_OUTPUT.PUT_LINE('Nu exista produsul cu acest cod!!');
   END IF;
   ROLLBACK;
END;
/
```




```
Nu exista produsul cu acest cod!!


PL/SQL procedure successfully completed.
```

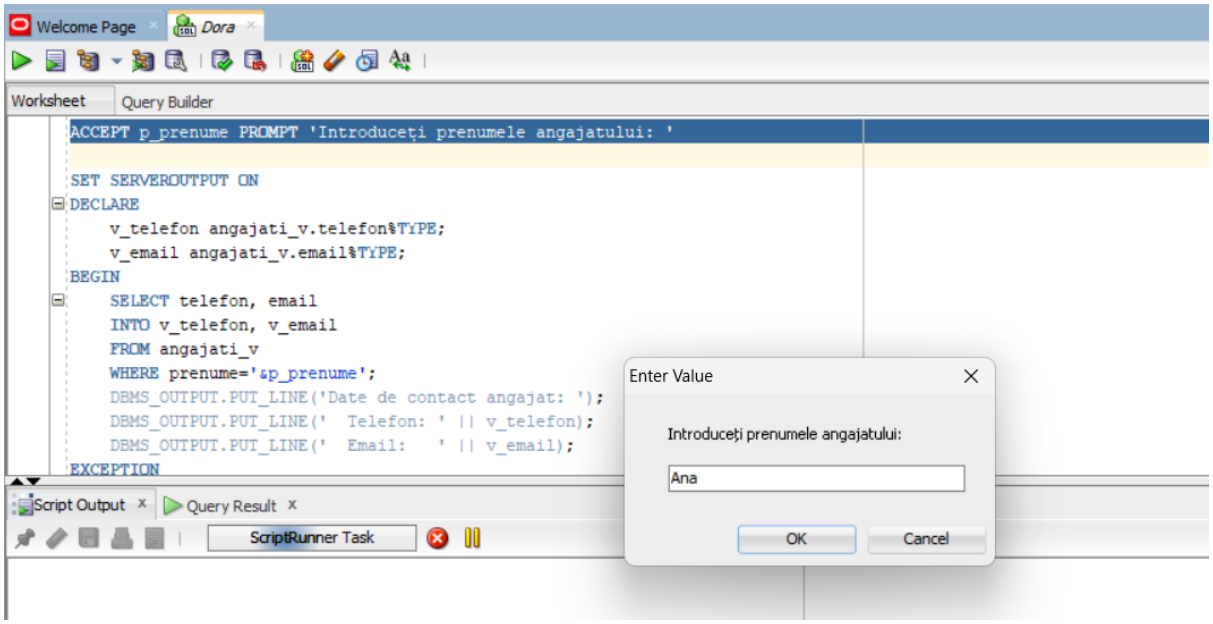
SEMINAR 6: tratarea excepțiilor

- Afișăm datele de contact ale unui angajat:

```
ACCEPT p_prenume PROMPT 'Introduceți prenumele angajatului: '

SET SERVEROUTPUT ON
DECLARE
   v_telefon angajati_v.telefon%TYPE;
   v_email angajati_v.email%TYPE;
BEGIN
   SELECT telefon, email
```

```
    INTO v_telefon, v_email
    FROM angajati_v
    WHERE prenume='&p_prenume';
    DBMS_OUTPUT.PUT_LINE('Date de contact angajat: ');
    DBMS_OUTPUT.PUT_LINE('  Telefon: ' || v_telefon);
    DBMS_OUTPUT.PUT_LINE('  Email:   ' || v_email);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Nu exista niciun angajat cu acest nume.');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Exista mai multi angajati cu acest nume.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('A aparut o eroare neasteptata.');
END;
/
```

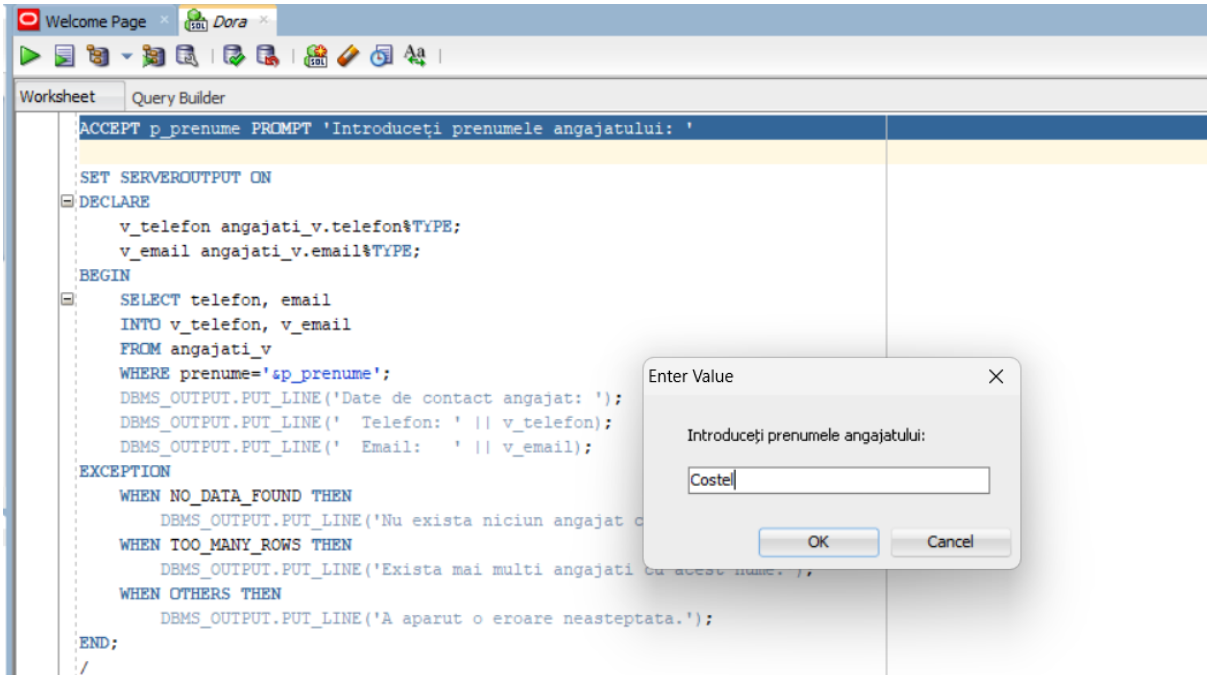


Când interogarea e corectă, se afișează:

```
Date de contact angajat:
  Telefon: 0735264387
  Email:   marinana65@gmail.com

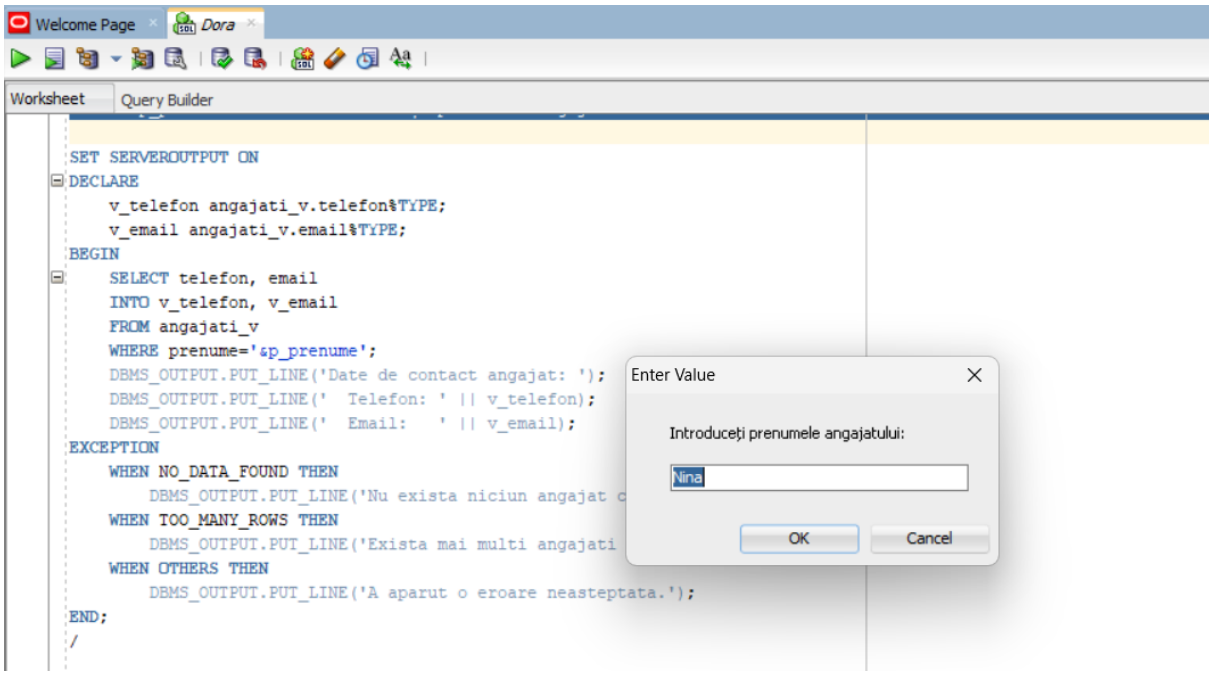

PL/SQL procedure successfully completed.
```

Altfel, se tratează excepția:




```
Exista mai multi angajati cu acest nume.


PL/SQL procedure successfully completed.
```




```
Nu exista niciun angajat cu acest nume.


PL/SQL procedure successfully completed.
```

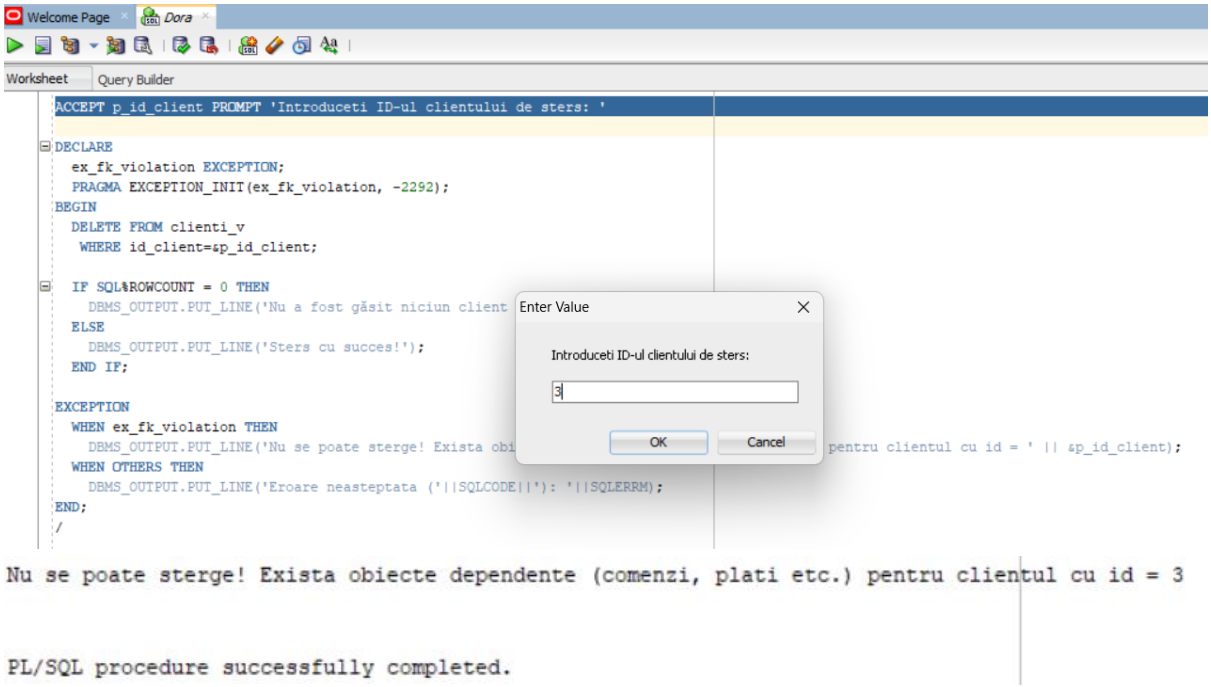
- Definim o excepție pentru ORA-02292 (FK violation) pentru atunci când se încearcă ștergerea unui client din baza de date:

```
ACCEPT p_id_client PROMPT 'Introduceti ID-ul clientului de sters: '

DECLARE
  ex_fk_violation EXCEPTION;
  PRAGMA EXCEPTION_INIT(ex_fk_violation, -2292);
BEGIN
  DELETE FROM clienti_v
   WHERE id_client=&p_id_client;

  IF SQL%ROWCOUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Nu a fost găsit niciun client cu acest id.');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Sters cu succes!');
  END IF;

EXCEPTION
  WHEN ex_fk_violation THEN
    DBMS_OUTPUT.PUT_LINE('Nu se poate sterge! Exista obiecte dependente (comenzi, plati etc.) pentru clientul cu id = ' || &p_id_client);
/
```



- Tratăm cazul în care se încearca inserarea în tabela produse, dar id-ul introdus este null, cu o excepție definită de utilizator:

```
DECLARE
  ex_null EXCEPTION;
  PRAGMA EXCEPTION_INIT(ex_null, -1400);
BEGIN
  INSERT INTO produse_v (id_produs, denumire_produs, descriere, categorie,
personalizabil)
  VALUES (NULL, 'Tiramisu', 'descriere', 'Torturi', 'NU');
  DBMS_OUTPUT.PUT_LINE('Produs inserat cu succes!');
EXCEPTION
  WHEN ex_null THEN
    DBMS_OUTPUT.PUT_LINE('EROARE!!! Coloana id_produs nu poate fi NULL!');
END;
/
```



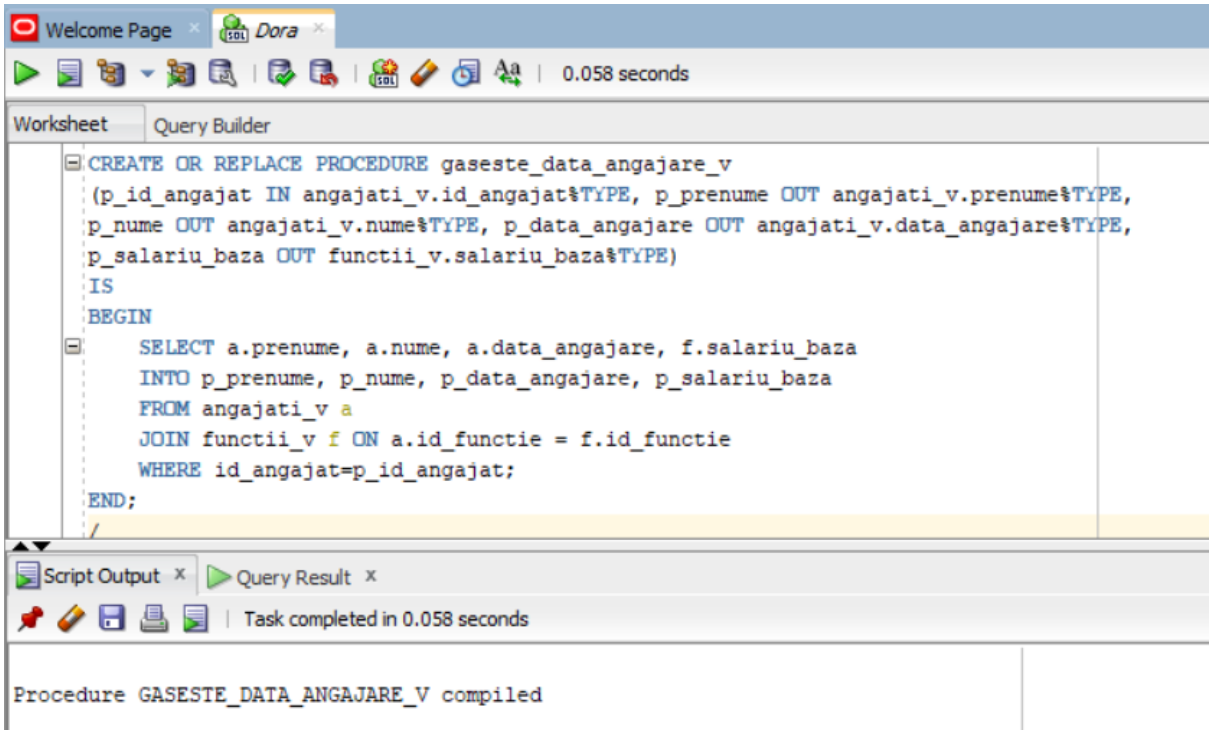
SEMINAR 7: subprograme PL/SQL

1. PROCEDURI

- Căutăm data de angajare a unui angajat cu o procedura (folosind parametrii de OUT):

```
CREATE OR REPLACE PROCEDURE gaseste_data_angajare_v
(p_id_angajat IN angajati_v.id_angajat%TYPE, p_prenume OUT
angajati_v.prenume%TYPE,
```

```
p_nume OUT angajati_v.nume%TYPE, p_data_angajare OUT
angajati_v.data_angajare%TYPE,
p_salariu_baza OUT functii_v.salariu_baza%TYPE)
IS
BEGIN
   SELECT a.prenume, a.nume, a.data_angajare, f.salariu_baza
   INTO p_prenume, p_nume, p_data_angajare, p_salariu_baza
   FROM angajati_v a
   JOIN functii_v f ON a.id_functie = f.id_functie
   WHERE id_angajat=p_id_angajat;
END;
/
```




```
SET SERVEROUTPUT ON
DECLARE
   v_prenume angajati_v.prenume%TYPE;
   v_nume angajati_v.nume%TYPE;
   v_data_angajare angajati_v.data_angajare%TYPE;
   v_salariu_baza functii_v.salariu_baza%TYPE;
BEGIN
   gaseste_data_angajare_v(11,v_prenume,v_nume,v_data_angajare,v_salariu_baza);
   DBMS_OUTPUT.PUT_LINE(v_prenume||' '||v_nume||' a fost angajat(a) pe data de
'||v_data_angajare||' cu un salariu initial de '||v_salariu_baza||' lei.');
END;
/
```



- Facem o procedura pentru a insera plati in tabela livrari_v pentru usurarea procesului

```
SET SERVEROUTPUT ON
CREATE OR REPLACE PROCEDURE inserare_in_livrari_v
(p_id_livrare number, p_id_comanda livrari_v.id_comanda%TYPE, p_adresa_livrare livrari_v.adresa_livrare%TYPE,
p_status_livrare livrari_v.status_livrare%TYPE, p_data_angajare date default sysdate)
IS
BEGIN
   INSERT INTO livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
   VALUES (p_id_livrare, p_id_comanda,p_adresa_livrare, p_status_livrare, p_data_angajare);
   DBMS_OUTPUT.PUT_LINE('Inserare efectuata!');
END;
/

SET SERVEROUTPUT ON
BEGIN
   inserare_in_livrari_v(11,12,'Bulevardul Dorobanti nr. 17','Livrata',SYSDATE);
END;
/
```

```
SET SERVEROUTPUT ON
CREATE OR REPLACE PROCEDURE inserare_in_livrari_v
(p_id_livrare number, p_id_comanda livrari_v.id_comanda%TYPE, p_adresa_livrare livrari_v.adresa_livrare%TYPE,
p_status_livrare livrari_v.status_livrare%TYPE, p_data_angajare date default sysdate)
IS
BEGIN
    INSERT INTO livrari_v (id_livrare, id_comanda, adresa_livrare, status_livrare, data_livrare)
    VALUES (p_id_livrare, p_id_comanda,p_adresa_livrare, p_status_livrare, p_data_angajare);
    DBMS_OUTPUT.PUT_LINE('Inserare efectuata!');
END;
/

SET SERVEROUTPUT ON
BEGIN
    inserare_in_livrari_v(11,12,'Bulevardul Dorobanti nr. 17','Livrata',SYSDATE);
END;
/
```

Script Output | Query Result

Task completed in 0.039 seconds

```
Procedure INSERARE_IN_LIVRARI_V compiled

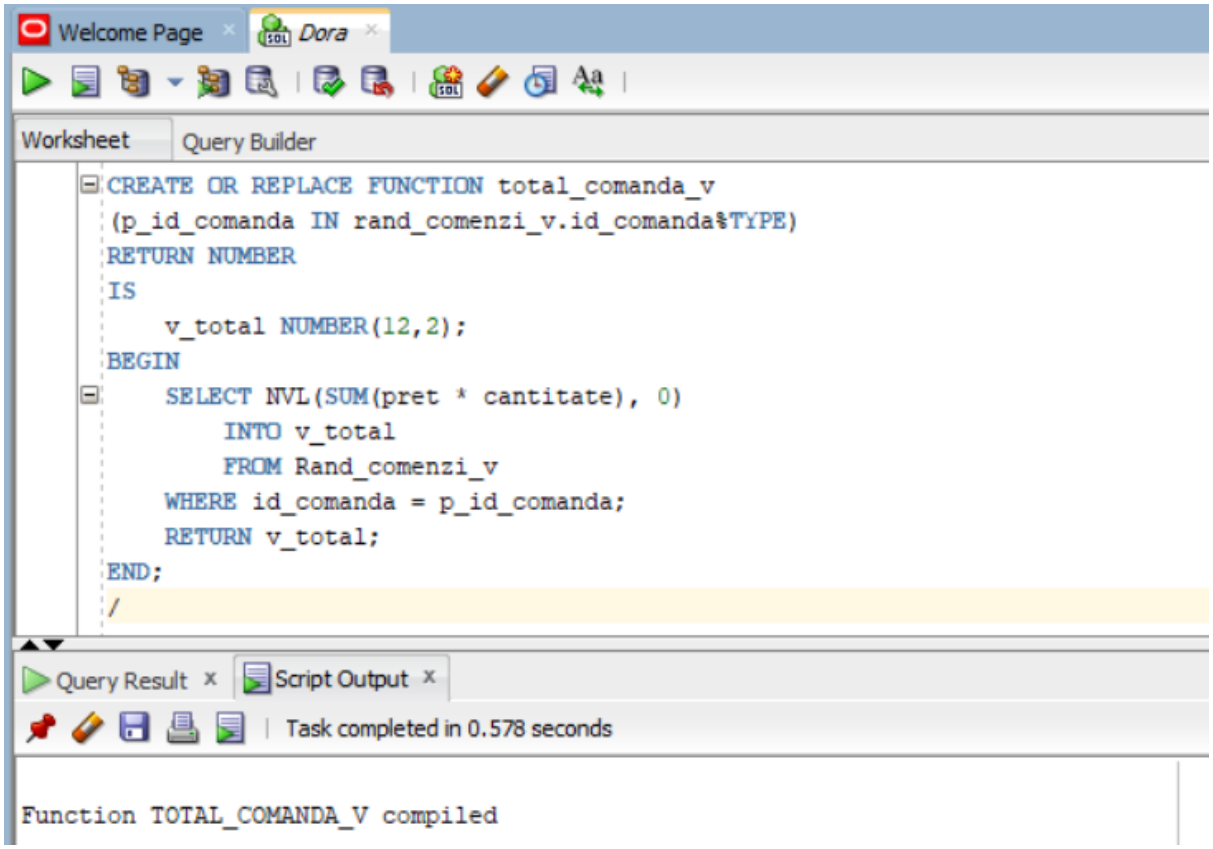
Inserare efectuata!


PL/SQL procedure successfully completed.
```

2. FUNCȚII

- Calculăm valoarea totală a unei comenzi (preț * cantitate pe fiecare linie):

```
CREATE OR REPLACE FUNCTION total_comanda_v
(p_id_comanda IN rand_comenzi_v.id_comanda%TYPE)
RETURN NUMBER
IS
   v_total NUMBER(12,2);
BEGIN
   SELECT NVL(SUM(pret * cantitate), 0)
      INTO v_total
      FROM Rand_comenzi_v
   WHERE id_comanda = p_id_comanda;
   RETURN v_total;
END;
/
```




```
SET SERVEROUTPUT ON
DECLARE
   v_id_comanda rand_comenzi_v.id_comanda%TYPE:=&p_id_comanda;
BEGIN
   DBMS_OUTPUT.PUT_LINE('Comanda cu id-ul '||v_id_comanda||' a costat
'||total_comanda_v(v_id_comanda)||' lei.');
END;
/
```

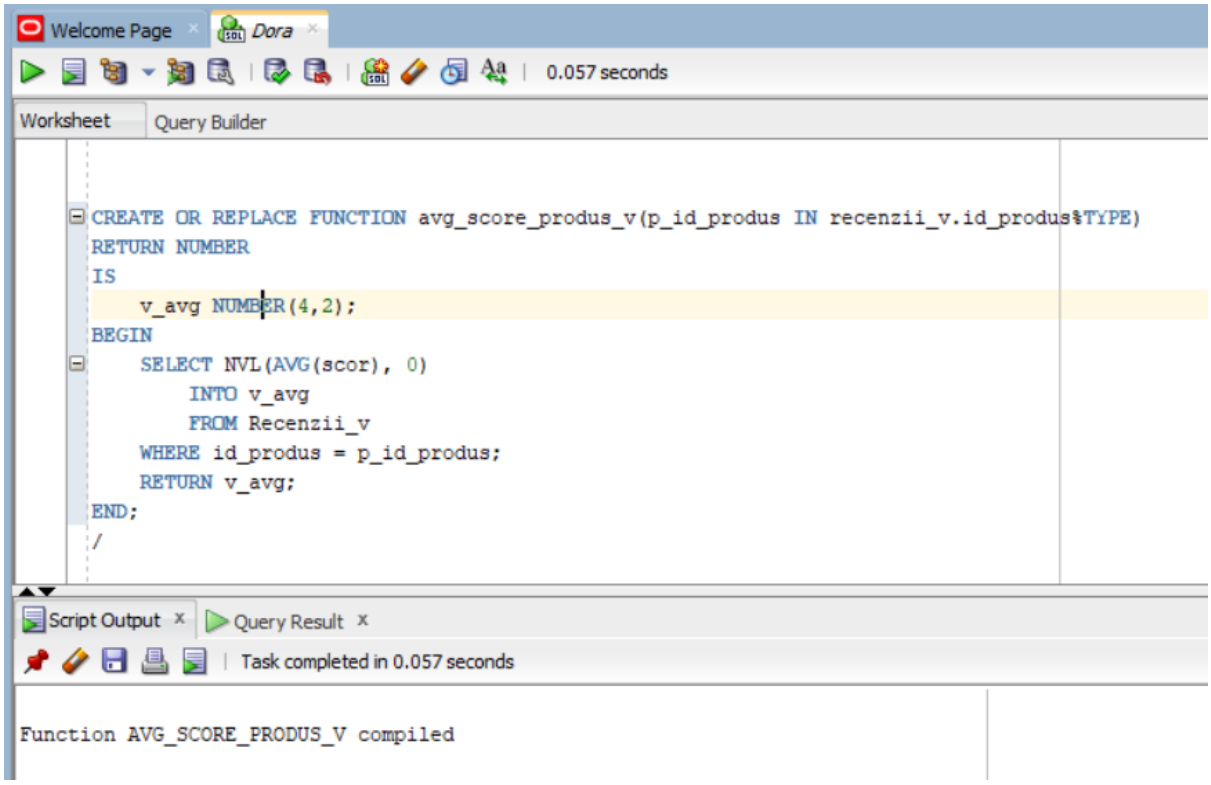
```
Comanda cu id-ul 7 a costat 300 lei.


PL/SQL procedure successfully completed.
```

- Calculăm scorul average al unui produs din tabela recenzii_v:

```
CREATE OR REPLACE FUNCTION avg_score_produs_v(p_id_produs IN
recenzii_v.id_produs%TYPE)
RETURN NUMBER
IS
   v_avg NUMBER(4,2);
BEGIN
   SELECT NVL(AVG(scor), 0)
```

```
        INTO v_avg
        FROM Recenzii_v
    WHERE id_produs = p_id_produs;
    RETURN v_avg;
END;
/
```




```
SET SERVEROUTPUT ON
DECLARE
    v_id_produs recenzii_v.id_produs%TYPE:=&p_id_produs;
    v_nume_produs produse_v.denumire_produs%TYPE;
BEGIN
    SELECT denumire_produs
    INTO v_nume_produs
    FROM produse_v
    WHERE id_produs=v_id_produs;
    DBMS_OUTPUT.PUT_LINE('Produsul '||lower(v_nume_produs)||' are ca scor mediu:
'||avg_score_produs_v(v_id_produs));
END;
/
```

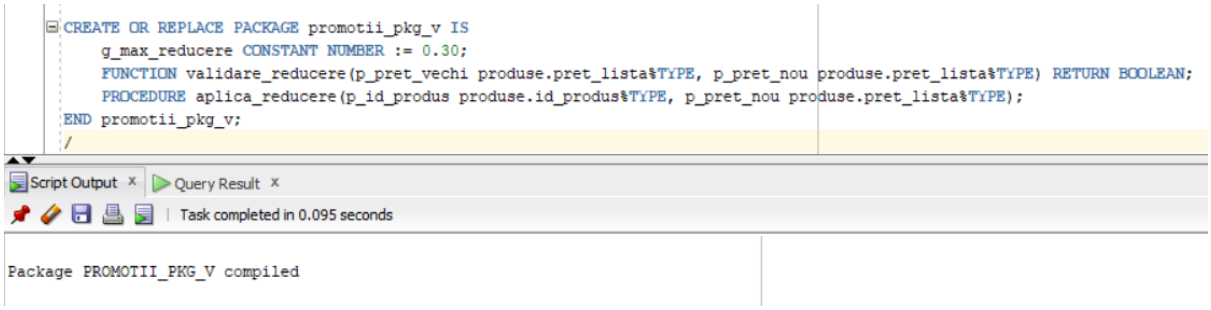
```
Produsul tiramisu are ca scor mediu: 9.75


PL/SQL procedure successfully completed.
```

SEMINAR 8: pachete de subprograme

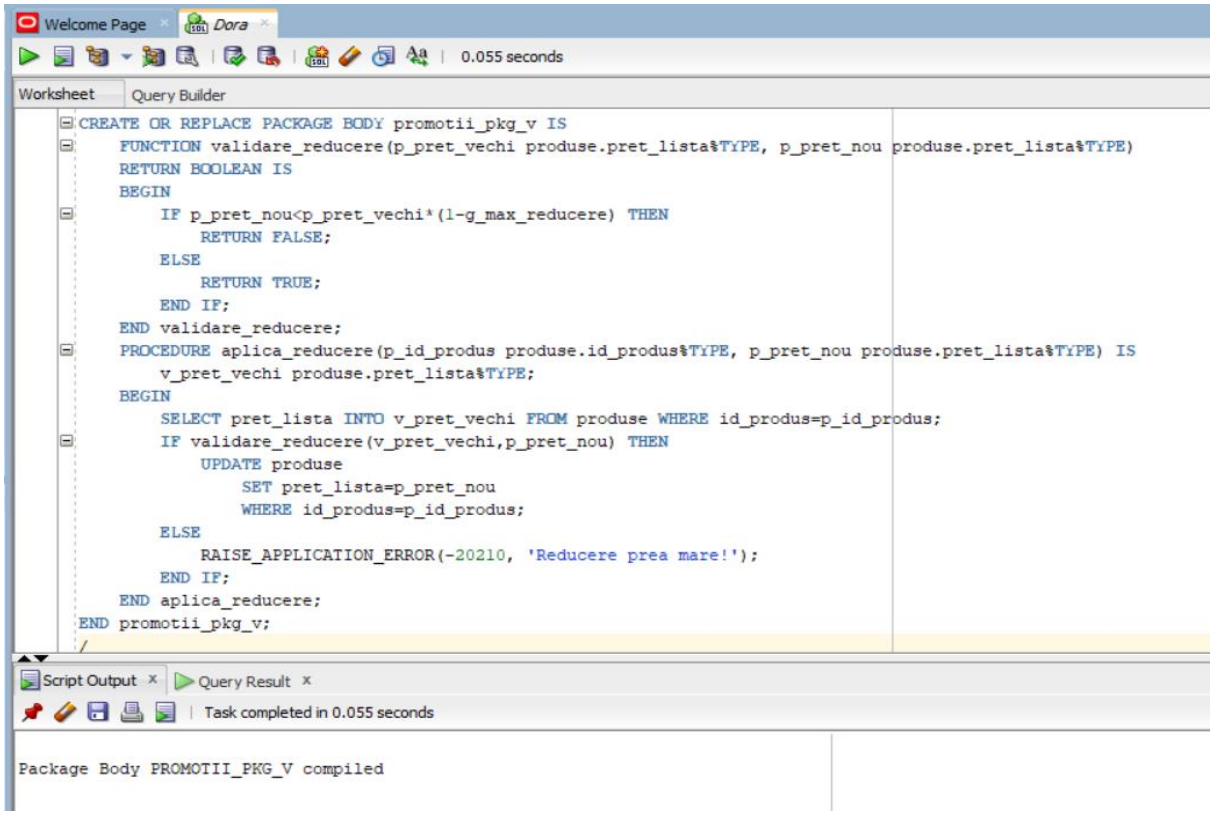
- Folosim un pachet pentru a face reduceri la produse, verificând daca diferența față de prețul inițial nu este prea drastică înainte:

```
CREATE OR REPLACE PACKAGE promotii_pkg_v IS
   g_max_reducere CONSTANT NUMBER := 0.30;
   FUNCTION validare_reducere(p_pret_vechi produse.pret_lista%TYPE, p_pret_nou
produse.pret_lista%TYPE) RETURN BOOLEAN;
   PROCEDURE aplica_reducere(p_id_produs produse.id_produs%TYPE, p_pret_nou
produse.pret_lista%TYPE);
END promotii_pkg_v;
/
```




```
CREATE OR REPLACE PACKAGE BODY promotii_pkg_v IS
   FUNCTION validare_reducere(p_pret_vechi produse.pret_lista%TYPE, p_pret_nou
produse.pret_lista%TYPE)
   RETURN BOOLEAN IS
   BEGIN
      IF p_pret_nou<p_pret_vechi*(1-g_max_reducere) THEN
         RETURN FALSE;
      ELSE
         RETURN TRUE;
      END IF;
   END validare_reducere;
   PROCEDURE aplica_reducere(p_id_produs produse.id_produs%TYPE, p_pret_nou
produse.pret_lista%TYPE) IS
      v_pret_vechi produse.pret_lista%TYPE;
   BEGIN
```

```
        SELECT pret_lista INTO v_pret_vechi FROM produse WHERE
id_produs=p_id_produs;
        IF validare_reducere(v_pret_vechi,p_pret_nou) THEN
           UPDATE produse
              SET pret_lista=p_pret_nou
              WHERE id_produs=p_id_produs;
        ELSE
           RAISE_APPLICATION_ERROR(-20210, 'Reducere prea mare!');
        END IF;
    END aplica_reducere;
END promotii_pkg_v;
/
```




```
SET SERVEROUTPUT ON
DECLARE
    v_denumire_produs produse.denumire_produs%TYPE;
    v_id_produs produse.id_produs%TYPE:=&v_id_produs;
    v_pret_nou produse.pret_lista%TYPE:=&p_pret_nou;
BEGIN
    SELECT denumire_produs INTO v_denumire_produs FROM produse WHERE
id_produs=v_id_produs;
   promotii_pkg_v.aplica_reducere(v_id_produs,v_pret_nou);
   DBMS_OUTPUT.PUT_LINE('Pretul produsului '||v_denumire_produs||' a fost redus.');
END;
```

Când funcționează:

```
Pretul produsului Spreadsheet - SSS/V 2.0 a fost redus.


PL/SQL procedure successfully completed.
```

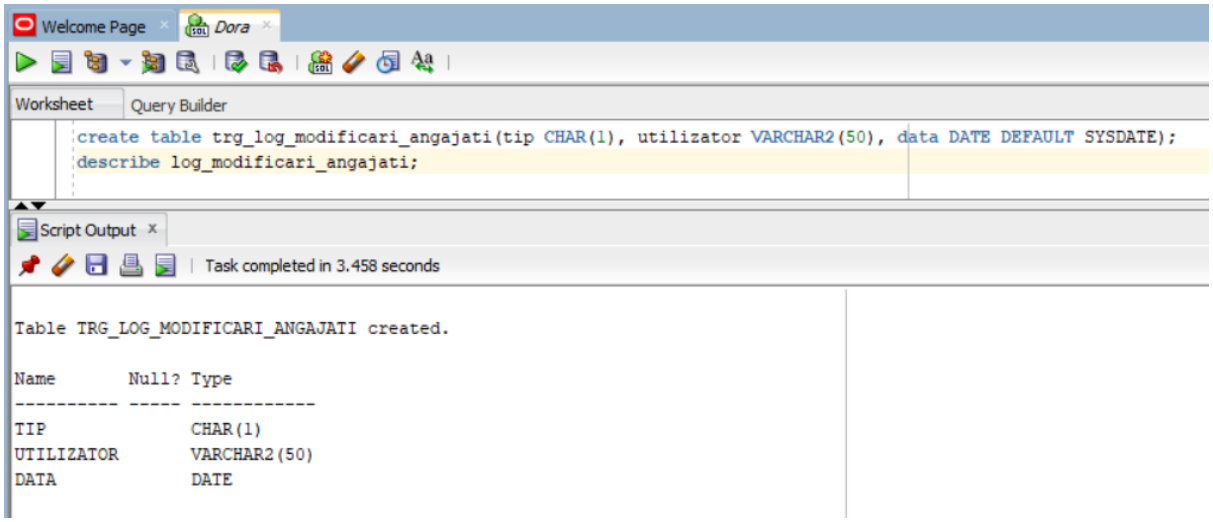
În cazul în care nu e corect:

```
ERROR at line 1:
ORA-20210: Reducere prea mare!
ORA-06512: at "CONDURACHED_56.PROMOTII_PKG_V", line 20
ORA-06512: at line 7
```

SEMINAR 9: triggeri pe baza de date

- Folosim o tabelă trig_log_modificari_angajati care să logheze modificările aduse angajaților cu ajutorul unui trigger:

create table trg_log_modificari_angajati(tip CHAR(1), utilizator VARCHAR2(50), data DATE DEFAULT SYSDATE);
describe log_modificari_angajati;



create or replace trigger trg_modif_angajati_v
before insert or update or delete on angajati_v
declare
v_tip log_modificari_angajati.tip%TYPE;
begin
case
when inserting then v_tip:='I';

when updating then v_tip:='U';
else v_tip:='D';
end case;
insert into trg_log_modificari_angajati(tip, utilizator, data) values (v_tip, user, sysdate);
end;
/

```
create or replace trigger trg_modif_angajati_v
before insert or update or delete on angajati_v
declare
v_tip log_modificari_angajati.tip%TYPE;
begin
case
when inserting then v_tip:='I';
when updating then v_tip:='U';
else v_tip:='D';
end case;
insert into trg_log_modificari_angajati(tip, utilizator, data) values (v_tip, user, sysdate);
end;
/
```

Script Output | Query Result | Task completed in 0.197 seconds

```
Trigger TRG_MODIF_ANGAJATI_V compiled
```

Facem niște modificări în tabela angajati_v pentru a testa:

insert into angajati_v
(id_angajat,prenume,nume,telefon,email,data_angajare,salariul,id_functie,id_manager)
values (18,'George','Washington','0782317533','gwash@gmail.com',to_date('30-JUL-2020','DD-MON-YYYY'),5000,8,10);

update angajati_v
set salariul=salariul+200
where id_angajat=18;

delete from angajati_v
where id_angajat=18;

```
insert into angajati_v (id_angajat,prenume,nume,telefon,email,data_angajare,salariul,id_functie,id_manager)
values (18,'George','Washington','0782317533','gwash@gmail.com',to_date('30-JUL-2020','DD-MON-YYYY'),5000,8,10);

update angajati_v
set salariul=salariul+200
where id_angajat=18;

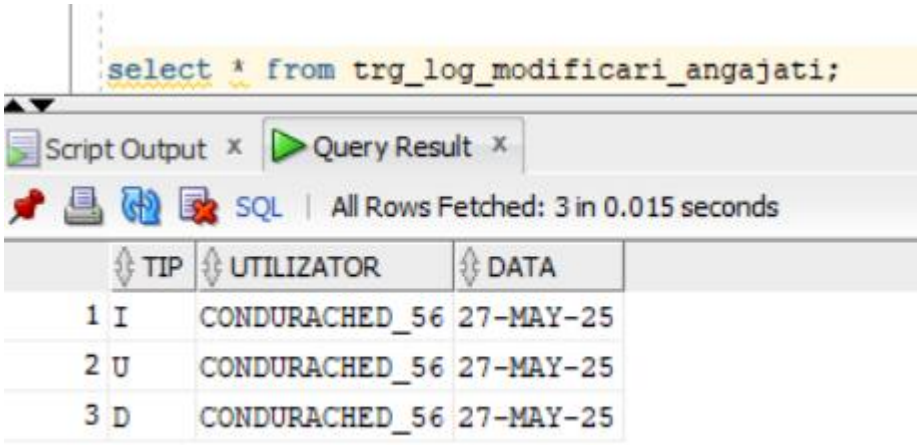
delete from angajati_v
where id_angajat=18;
```

Script Output | Query Result | Task completed in 0.027 seconds

```
1 row inserted.

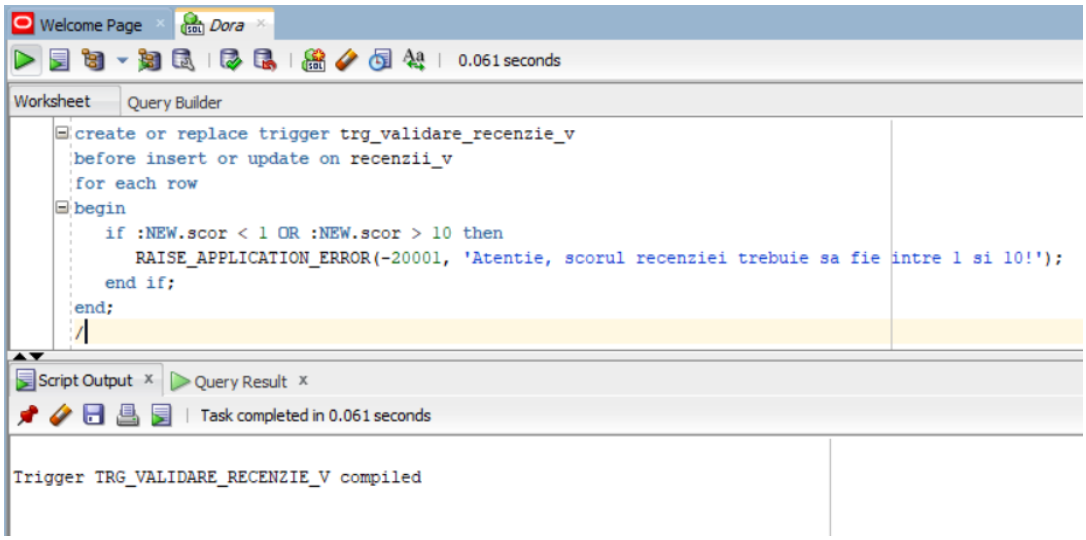

1 row updated.


1 row deleted.
```



- Folosim un trigger pentru a atenționa dacă scorul introdus nu este valid (adică e <1 sau >10)

```
create or replace trigger trg_validare_recenzie_v
before insert or update on recenzii_v
for each row
begin
  if :NEW.scor < 1 OR :NEW.scor > 10 then
    RAISE_APPLICATION_ERROR(-20001, 'Atentie, scorul recenziei trebuie sa fie intre 1 si 10!');
  end if;
end;
/
```



Acum testarea cu o încercare de inserare în recenzii_v:

```
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (17, 3, 11, TO_DATE('2024-12-01', 'YYYY-MM-DD'));
```

```
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (17, 3, 11, TO_DATE('2024-12-01', 'YYYY-MM-DD'));
```

Script Output x | Query Result x

Task completed in 0.129 seconds

```
Trigger TRG_VALIDARE_RECENZIE_V compiled


Error starting at line : 13 in command -
INSERT INTO Recenzii_v (id_recenzie, id_produs, scor, data_inregistrare)
VALUES (17, 3, 11, TO_DATE('2024-12-01', 'YYYY-MM-DD'))
Error at Command Line : 13 Column : 13
Error report -
SQL Error: ORA-20001: Atentie, scorul recenziei trebuie sa fie intre 1 si 10!
ORA-06512: at "CONDURACHED_56.TRG_VALIDARE_RECENZIE_V", line 3
ORA-04088: error during execution of trigger 'CONDURACHED_56.TRG_VALIDARE_RECENZIE_V'
```